Algoritmos e Sistemas Dinâmicos em Grafos

Relatório Final de Iniciação Científica

Nome do aluno: Gabriel Oliveira dos Santos

Orientador: Gustavo Henrique Labegalini Boska

12 de novembro de 2026

Resumo

Este trabalho apresenta o estudo de conceitos fundamentais da teoria dos grafos e sua aplicação à modelagem de sistemas dinâmicos em redes. São abordados algoritmos de manipulação e busca em grafos, árvores geradoras mínimas, matrizes de representação e propriedades espectrais, com destaque para a matriz Laplaciana. A investigação inclui a construção e análise de uma rede aleatória com duas comunidades e a simulação da difusão de calor sobre seus vértices por meio do método de Euler explícito. Os resultados obtidos incluem a análise de conectividade e do espectro da matriz Laplaciana, além da observação da evolução temporal das temperaturas. Também são comparados dois experimentos com diferentes valores da constante de difusão: $c = 0{,}01$, no primeiro vídeo, e $c = 0{,}08$, no segundo. A comparação evidencia qualitativamente que valores maiores de c promovem uma equalização térmica mais rápida, respeitada a condição de estabilidade do método numérico. Os resultados demonstram como a estrutura da rede e os parâmetros do modelo influenciam a dinâmica de difusão.

Palavras-chave: teoria dos grafos; algoritmos em grafos; árvores geradoras; teoria espectral de grafos; matriz Laplaciana; sistemas dinâmicos; difusão em redes.

1 Introdução

A teoria dos grafos fornece uma linguagem matemática e computacional para representar sistemas formados por elementos e relações entre esses elementos. Redes de comunicação, sistemas de transporte, interações sociais e estruturas de infraestrutura são exemplos de situações que podem ser modeladas por vértices e arestas. Além da representação estática dessas relações, a estrutura de um grafo também pode determinar como uma quantidade evolui ao longo do tempo sobre a rede.

Este trabalho está organizado em duas frentes complementares. A primeira concentra-se no estudo de estruturas e algoritmos em grafos, incluindo representações computacionais, buscas, conectividade, árvores, árvores geradoras e árvores geradoras de peso mínimo. A segunda utiliza a matriz Laplaciana para relacionar a estrutura da rede a sistemas dinâmicos, com ênfase em um modelo de difusão.

O trabalho combina fundamentação matemática e implementação computacional em Python. Foram desenvolvidas as estruturas e operações básicas sobre grafos, os algoritmos de busca e conectividade, a construção de árvores geradoras de peso mínimo, as matrizes associadas aos grafos e a análise espectral da matriz Laplaciana. Essa base foi utilizada na modelagem e na simulação de um sistema de difusão em uma rede com duas comunidades. Dessa forma, o estudo estabelece uma relação entre a estrutura do grafo, sua representação matricial, os algoritmos implementados e o comportamento dinâmico observado.

1.1 Objetivos

O objetivo geral deste trabalho é estudar como estruturas matemáticas e algoritmos de grafos podem ser utilizados para representar, analisar e modelar sistemas dinâmicos em redes.

Como objetivos específicos, pretende-se:

- estudar representações computacionais de grafos;
- estudar algoritmos de busca, com ênfase em BFS e DFS;
- analisar conectividade, componentes conexas, árvores e árvores geradoras;
- estudar árvores geradoras de peso mínimo e o algoritmo de Kruskal;
- construir e analisar matrizes de adjacência, de graus e Laplaciana;
- estudar propriedades espectrais da matriz Laplaciana;
- modelar sistemas dinâmicos definidos sobre grafos;
- implementar e analisar numericamente um modelo de difusão;
- realizar experimentos em redes com diferentes estruturas, incluindo redes com comunidades;
- investigar a relação entre características estruturais, propriedades espectrais e comportamento dinâmico.

2 Fundamentação teórica em grafos

Esta seção apresenta os principais conceitos matemáticos e computacionais necessários para o desenvolvimento do trabalho. São introduzidas as definições fundamentais de grafos, caminhos, conectividade, representações computacionais, árvores, árvores geradoras, árvores geradoras de peso mínimo, algoritmos de busca, algoritmo de Kruskal e matriz Laplaciana.

A fundamentação apresentada nesta seção fornece a base para a modelagem posterior de sistemas dinâmicos definidos sobre grafos, em especial para o estudo de processos de difusão em redes.

Os conceitos fundamentais de teoria dos grafos utilizados ao longo do trabalho são baseados principalmente em [3], enquanto os aspectos relacionados à matriz Laplaciana e à análise de redes são complementados por [2] e [1].

2.1 Grafos, caminhos e conectividade

Um grafo é uma estrutura matemática utilizada para representar relações entre objetos. Formalmente, um grafo simples não direcionado é definido por

$$G = (V, E),$$

onde V representa o conjunto de vértices e E representa o conjunto de arestas.

Os elementos de V são denominados vértices ou nós, enquanto os elementos de E representam as conexões existentes entre pares de vértices.

Por exemplo, considere

$$V = \{0, 1, 2, 3\}$$

e

$$E = \{\{0, 1\}, \{0, 2\}, \{1, 2\}, \{1, 3\}\}.$$

Nesse caso, o grafo possui quatro vértices e quatro arestas.

Os principais conceitos utilizados neste trabalho são:

Grafo: Estrutura matemática composta por um conjunto de vértices e um conjunto de arestas, utilizada para representar relações entre objetos.

Vértice: Unidade fundamental de um grafo, representando um ponto, objeto ou entidade.

Aresta: Ligação que conecta dois vértices e representa uma relação direta entre eles.

Grau: Número de arestas incidentes em um determinado vértice. Em um grafo simples não direcionado, corresponde ao número de vizinhos desse vértice.

Vizinhos: Dois vértices são considerados vizinhos quando existe uma aresta conectando-os.

Passeio: Sequência de vértices em que cada par consecutivo está conectado por uma aresta. Um passeio pode, em geral, repetir vértices e arestas.

Caminho simples: Passeio no qual nenhum vértice é repetido.

Ciclo: Passeio fechado no qual o vértice inicial e o final coincidem e não há repetição dos vértices intermediários.

Grafo conexo: Grafo no qual existe um caminho entre qualquer par de vértices.

Componente conexa: Subgrafo conexo maximal de um grafo. Em uma componente conexa, existe um caminho entre quaisquer dois de seus vértices.

A Figura 1 apresenta um exemplo de grafo, permitindo visualizar a relação entre vértices e arestas.

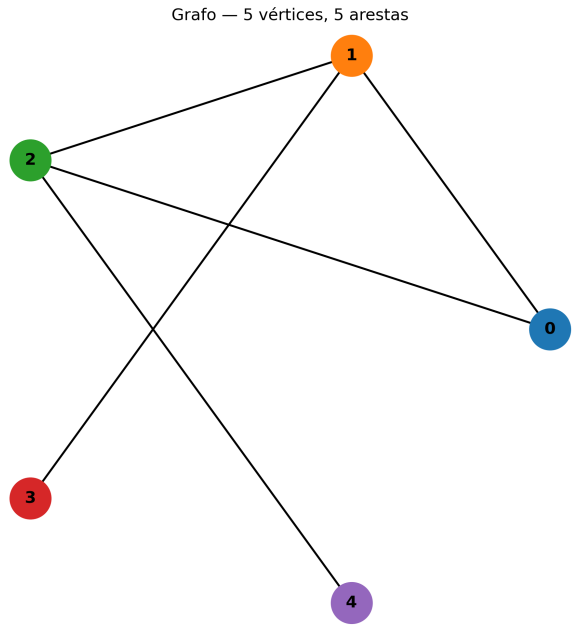


Figura 1: Exemplo de um grafo com seus vértices e arestas.

A conectividade é uma propriedade importante para o desenvolvimento deste trabalho, pois vários dos algoritmos estudados dependem da possibilidade de alcançar diferentes vértices a partir de um vértice inicial.

Em particular, os algoritmos de busca BFS e DFS permitem explorar sistematicamente os vértices alcançáveis de um grafo.

2.2 Representações computacionais de grafos

Para utilizar grafos em programas de computador, é necessário escolher uma estrutura de dados capaz de representar seus vértices e arestas.

Entre as representações mais utilizadas estão as listas de adjacência e as matrizes de adjacência.

Lista de adjacência: Estrutura que associa a cada vértice uma lista contendo seus vizinhos. Essa representação é particularmente adequada para grafos esparsos, nos quais o número de arestas é relativamente pequeno em comparação com o número máximo possível de arestas.

Matriz de adjacência: Matriz quadrada na qual linhas e colunas representam os vértices do grafo. A posição (i, j) indica a existência de uma aresta entre os vértices i e j, ou pode armazenar seu peso em um grafo ponderado.

Na representação por lista de adjacência, um grafo pode ser representado, por exemplo, por

$$\begin{aligned} &0 : [1, 2], \\ &1 : [0, 2, 3], \\ &2 : [0, 1], \\ &3 : [1]. \end{aligned}$$

Essa representação corresponde ao grafo utilizado como exemplo anteriormente.

No presente trabalho, a implementação computacional utiliza uma estrutura baseada em lista de adjacência. Essa escolha permite representar diretamente os vizinhos de cada vértice e facilita a implementação dos algoritmos de busca e das operações básicas sobre grafos.

2.3 Matriz de adjacência

Seja

$$G = (V, E)$$

um grafo com

$$V = \{v_1, v_2, \ldots, v_n\}.$$

A matriz de adjacência A de G é uma matriz quadrada de dimensão $n \times n$. Para um grafo simples não direcionado, cada elemento A_{ij} é definido por

$$A_{ij} = \begin{cases} 1, & \text{se } \{v_i, v_j\} \in E, \\ 0, & \text{caso contrário.} \end{cases}$$

Em um grafo não direcionado, a existência da aresta $\{v_i, v_j\}$ implica a existência da mesma conexão no sentido oposto. Portanto,

$$A_{ij} = A_{ji},$$

fazendo com que

$$A = A^T.$$

A matriz de adjacência fornece uma representação algébrica da estrutura do grafo. A partir dela, diversas propriedades podem ser estudadas por meio de operações matriciais.

Uma propriedade particularmente importante é a interpretação das potências da matriz de adjacência.

Para um inteiro positivo k, a entrada

$$(A^k)_{ij}$$

representa o número de passeios de comprimento k que começam no vértice v_i e terminam no vértice v_j.

2.3.1 Demonstração da interpretação de A^k

A propriedade pode ser demonstrada por indução matemática sobre o comprimento k do passeio.

Caso base ($k = 1$):

Para $k = 1$,

$$A^1 = A.$$

Pela definição da matriz de adjacência,

$$A_{ij} = 1$$

quando existe uma aresta entre v_i e v_j, e

$$A_{ij} = 0$$

caso contrário.

Portanto, A_{ij} representa o número de passeios de comprimento 1 entre os vértices.

Passo indutivo:

Suponha que, para um determinado inteiro $k - 1 \geq 1$, $(A^{k-1})_{im}$ represente corretamente o número de passeios de comprimento $k - 1$ entre v_i e v_m.

Um passeio de comprimento k entre v_i e v_j pode ser dividido em duas partes:

1. um passeio de comprimento $k - 1$ entre v_i e algum vértice intermediário v_m;
2. uma última aresta conectando v_m a v_j.

Para um determinado vértice intermediário v_m, a quantidade de possibilidades é

$$(A^{k-1})_{im} A_{mj}.$$

Somando sobre todos os possíveis vértices intermediários, temos

$$(A^k)_{ij} = \sum_{m=1}^{n} (A^{k-1})_{im} A_{mj}.$$

Essa expressão corresponde exatamente à definição da multiplicação de matrizes:

$$A^k = A^{k-1} A.$$

Portanto, $(A^k)_{ij}$ contabiliza todos os passeios de comprimento k que começam em v_i e terminam em v_j.

2.3.2 Exemplo

Considere o grafo com vértices A, B, C e D, e arestas

$$\{A, B\}, \quad \{A, C\}, \quad \{B, D\}, \quad \{C, D\}.$$

Sua matriz de adjacência é

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix}.$$

Calculando A^2, obtemos

$$A^2 = \begin{pmatrix} 2 & 0 & 0 & 2 \\ 0 & 2 & 2 & 0 \\ 0 & 2 & 2 & 0 \\ 2 & 0 & 0 & 2 \end{pmatrix}.$$

A entrada

$$(A^2)_{11} = 2$$

indica que existem dois passeios de comprimento 2 que começam e terminam em A:

$$A \to B \to A$$

e

$$A \to C \to A.$$

Essa interpretação mostra como propriedades combinatórias do grafo podem ser analisadas por meio de operações matriciais.

2.4 Árvores e árvores geradoras

Uma **árvore** é um grafo não direcionado, conexo e acíclico. Equivalentemente, um grafo $T = (V, E)$ é uma árvore quando existe exatamente um caminho simples entre quaisquer dois de seus vértices.

As árvores são estruturas fundamentais na teoria dos grafos, pois representam formas de conexão sem ciclos e sem redundância de arestas.

2.4.1 Propriedades básicas das árvores

Uma árvore com n vértices possui exatamente

$$n - 1$$

arestas.

Além disso, uma árvore possui as seguintes propriedades:

- é conexa;

- não possui ciclos;
- existe exatamente um caminho simples entre quaisquer dois vértices;
- se possui n vértices, possui $n - 1$ arestas;
- se $n \geq 2$, possui pelo menos duas folhas, isto é, vértices de grau 1.

Outra propriedade importante é a minimalidade da conectividade. Se uma aresta qualquer de uma árvore for removida, o grafo resultante deixa de ser conexo.

Por outro lado, se uma nova aresta for adicionada entre dois vértices de uma árvore, será criado um ciclo.

Essas características fazem das árvores estruturas importantes para algoritmos de busca e para problemas de conexão mínima.

2.4.2 Árvores geradoras

Considere um grafo conexo

$$G = (V, E).$$

Uma **árvore geradora** (*spanning tree*) de G é um subgrafo

$$T = (V, E_T)$$

que contém todos os vértices de G e que é uma árvore.

Assim, uma árvore geradora preserva todos os vértices do grafo original, mas utiliza apenas parte das suas arestas.

Como toda árvore com n vértices possui $n - 1$ arestas, toda árvore geradora satisfaz

$$|E_T| = |V| - 1.$$

Todo grafo conexo possui pelo menos uma árvore geradora. Uma forma natural de obter uma árvore geradora é utilizar as arestas de descoberta produzidas por uma busca BFS ou DFS.

Um mesmo grafo pode possuir diversas árvores geradoras. Isso ocorre porque diferentes subconjuntos de arestas podem conectar todos os vértices sem formar ciclos.

2.4.3 Relação com algoritmos de busca

As árvores geradoras aparecem naturalmente durante a execução de algoritmos de busca.

Na **Busca em Largura** (*Breadth First Search* – BFS), os vértices são explorados por níveis, enquanto na **Busca em Profundidade** (*Depth First Search* – DFS), a exploração segue o caminho mais profundamente possível antes de retornar.

Durante ambas as buscas, cada vértice novo é associado à aresta pela qual foi descoberto.

Considere uma BFS iniciada em v_0. Inicialmente temos

$$T_0 = (\{v_0\}, \emptyset).$$

Quando um novo vértice v é descoberto a partir de um vértice u, a aresta

$$\{u, v\}$$

é adicionada à estrutura de busca.

Como v ainda não havia sido visitado, a nova aresta conecta um vértice novo à estrutura existente sem criar um ciclo.

Repetindo esse processo, as arestas utilizadas para a descoberta dos vértices formam uma árvore de busca.

Se o grafo original for conexo, todos os seus vértices serão alcançados. Nesse caso, a árvore de busca contém todos os vértices do grafo e, portanto, é uma árvore geradora.

A mesma ideia se aplica à DFS, embora a ordem de exploração seja diferente.

2.5 Árvores geradoras de peso mínimo

Em muitos problemas de redes, as conexões entre os vértices possuem custos associados.

Um **grafo ponderado** é um grafo no qual cada aresta possui um valor denominado peso. Esse peso pode representar, por exemplo, distância, custo, tempo ou outro tipo de medida associada à conexão.

Podemos representar o peso de uma aresta e por

$$w(e).$$

Considere novamente um grafo conexo

$$G = (V, E).$$

Entre todas as suas árvores geradoras, podemos procurar aquela cujo peso total seja mínimo.

Se

$$T = (V, E_T)$$

é uma árvore geradora ponderada, seu peso total é definido por

$$w(T) = \sum_{e \in E_T} w(e).$$

Uma árvore geradora T é denominada uma **árvore geradora mínima** (*Minimum Spanning Tree* – MST) quando

$$w(T) \leq w(T')$$

para toda árvore geradora T' de G.

O objetivo do problema da árvore geradora mínima é, portanto, conectar todos os vértices do grafo utilizando o menor peso total possível e sem formar ciclos.

Esse problema possui diversas aplicações em redes de comunicação, infraestrutura, distribuição e planejamento de conexões.

2.6 Algoritmo de Kruskal

Um dos algoritmos utilizados para encontrar uma árvore geradora mínima é o **algoritmo de Kruskal**.

O algoritmo utiliza uma estratégia denominada **gulosa** (*greedy*), pois analisa as arestas em ordem crescente de peso e procura tomar, a cada etapa, uma escolha local que possa fazer parte da solução final.

Inicialmente, cada vértice é considerado uma componente separada. As arestas são então ordenadas de acordo com seus pesos:

$$w(e_1) \leq w(e_2) \leq \cdots \leq w(e_m).$$

Em seguida, as arestas são analisadas nessa ordem.

Para cada aresta considerada, existem duas possibilidades:

- se os extremos da aresta pertencem a componentes diferentes, a aresta é adicionada à árvore;
- se os extremos pertencem à mesma componente, a aresta é descartada, pois sua inclusão produziria um ciclo.

Portanto, a regra fundamental de Kruskal pode ser resumida como:

> Adicionar a aresta de menor peso disponível que não produza um ciclo.

A verificação eficiente das componentes pode ser realizada por uma estrutura de conjuntos disjuntos, conhecida como *Union-Find* ou *Disjoint Set Union* (DSU).

O algoritmo termina quando a árvore possui

$$|V| - 1$$

arestas.

Como cada aresta adicionada conecta duas componentes diferentes, não são formados ciclos. Ao final, todas as componentes estão conectadas e a estrutura resultante é uma árvore.

A estratégia gulosa utilizada pelo algoritmo está relacionada ao princípio do corte. Considere uma divisão dos vértices em duas partes e as arestas que conectam essas partes. A aresta de menor peso que atravessa esse corte pode ser escolhida como parte de uma árvore geradora mínima.

Esse princípio fornece a base para as escolhas realizadas pelo algoritmo de Kruskal.

A implementação do algoritmo será apresentada posteriormente na seção dedicada à implementação computacional.

2.7 Difusão de calor em grafos

Neste trabalho, o grafo é utilizado para representar uma rede na qual o calor pode se distribuir entre os vértices. Cada vértice representa uma posição da rede na qual existe uma determinada temperatura, enquanto as arestas representam as conexões entre essas posições. Dessa forma, o calor pode ser transferido diretamente entre vértices que estão conectados por uma aresta.

Considere um grafo com n vértices. Para cada vértice v_i, seja

$$u_i(t)$$

a temperatura do vértice no instante t.

O estado térmico completo da rede pode ser representado pelo vetor

$$u(t) = \begin{bmatrix} u_1(t) \\ u_2(t) \\ \vdots \\ u_n(t) \end{bmatrix}.$$

A temperatura de cada vértice pode variar de acordo com as temperaturas dos vértices diretamente conectados a ele. Quando existe uma diferença de temperatura entre dois vértices conectados, ocorre uma tendência de transferência de calor entre eles.

Assim, a variação da temperatura de um vértice depende das diferenças entre sua própria temperatura e as temperaturas de seus vizinhos.

Considere inicialmente um vértice v_i. A contribuição dos seus vizinhos para a variação de sua temperatura pode ser expressa por

$$\frac{du_i(t)}{dt} = c \sum_{j \sim i} \left(u_j(t) - u_i(t) \right),$$

onde $c > 0$ representa uma constante associada à taxa de difusão do calor e $j \sim i$ indica que os vértices v_i e v_j estão conectados por uma aresta.

Essa equação mostra diretamente o comportamento esperado da difusão. Se a temperatura de um vértice for maior que a temperatura de seus vizinhos, a diferença tende a produzir uma redução de sua temperatura. Por outro lado, se sua temperatura for menor que a de seus vizinhos, essa contribuição tende a aumentar sua temperatura.

Portanto, o processo tende a reduzir as diferenças de temperatura entre vértices conectados, levando a rede em direção a um estado de equilíbrio térmico.

Para escrever esse sistema de forma matricial, é necessário representar simultaneamente as conexões entre os vértices e o número de vizinhos de cada vértice. Para isso, introduz-se a matriz Laplaciana do grafo.

A matriz Laplaciana é definida por

$$L = D - A,$$

onde A é a matriz de adjacência do grafo e D é a matriz diagonal dos graus dos vértices.

A matriz de graus possui a forma

$$D = \begin{bmatrix} d_1 & 0 & \cdots & 0 \\ 0 & d_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & d_n \end{bmatrix},$$

onde

$$d_i = \deg(v_i).$$

A matriz de adjacência A indica quais vértices estão conectados, enquanto a matriz D registra quantos vizinhos cada vértice possui. A combinação dessas duas matrizes permite representar as diferenças de temperatura entre cada vértice e seus vizinhos.

A partir da matriz Laplaciana, o sistema de difusão de calor pode ser escrito na forma

$$\frac{du(t)}{dt} = -cLu(t).$$

Nessa equação, $u(t)$ representa o vetor de temperaturas da rede, $\frac{du(t)}{dt}$ representa a taxa de variação dessas temperaturas, c representa a constante de difusão e L representa a matriz Laplaciana do grafo.

A componente correspondente ao vértice i pode ser obtida a partir de

$$(Lu(t))_i = \sum_{j \sim i} \left(u_i(t) - u_j(t)\right).$$

Consequentemente,

$$\frac{du_i(t)}{dt} = -c \sum_{j \sim i} \left(u_i(t) - u_j(t)\right),$$

ou, de forma equivalente,

$$\frac{du_i(t)}{dt} = c \sum_{j \sim i} \left(u_j(t) - u_i(t)\right).$$

Essa formulação mostra a relação entre a estrutura do grafo e a dinâmica térmica. As arestas determinam quais vértices podem trocar calor diretamente, enquanto a matriz Laplaciana reúne essas conexões em uma representação matricial que pode ser utilizada para descrever a evolução das temperaturas ao longo do tempo.

2.7.1 Exemplo de transferência de calor

Considere dois vértices conectados, com temperaturas

$$u_1(t) = 100^\circ\text{C}$$

e

$$u_2(t) = 20^\circ\text{C}.$$

Supondo, para simplificar, que $c = 1$, para o primeiro vértice temos

$$\frac{du_1(t)}{dt} = u_2(t) - u_1(t).$$

Substituindo os valores,

$$\frac{du_1(t)}{dt} = 20 - 100 = -80.$$

Como a derivada é negativa, a temperatura do primeiro vértice tende a diminuir. Para o segundo vértice,

$$\frac{du_2(t)}{dt} = u_1(t) - u_2(t).$$

Logo,

$$\frac{du_2(t)}{dt} = 100 - 20 = 80.$$

Como a derivada é positiva, a temperatura do segundo vértice tende a aumentar.

Portanto, inicialmente, o vértice mais quente perde calor enquanto o vértice mais frio ganha calor. Esse comportamento representa a tendência de transferência de calor das regiões de maior temperatura para as regiões de menor temperatura.

2.7.2 Influência de vários vizinhos

Quando um vértice possui vários vizinhos, sua variação de temperatura depende da contribuição de todos eles.

Por exemplo, considere um vértice i conectado aos vértices 1, 2 e 3. Nesse caso,

$$\frac{du_i(t)}{dt} = c\left[(u_1(t) - u_i(t)) + (u_2(t) - u_i(t)) + (u_3(t) - u_i(t))\right].$$

Assim, não é necessário que todos os vizinhos sejam mais quentes ou mais frios. O que determina se o vértice ganha ou perde calor é o resultado da soma das diferenças de temperatura.

Se

$$\sum_{j\sim i}(u_j(t) - u_i(t)) > 0,$$

então

$$\frac{du_i(t)}{dt} > 0,$$

e o vértice i tende a ganhar calor.

Se

$$\sum_{j\sim i}(u_j(t) - u_i(t)) < 0,$$

então

$$\frac{du_i(t)}{dt} < 0,$$

e o vértice i tende a perder calor.

Por fim, se

$$\sum_{j\sim i}(u_j(t) - u_i(t)) = 0,$$

então

$$\frac{du_i(t)}{dt} = 0,$$

e a temperatura do vértice não se altera naquele instante.

2.7.3 Equilíbrio térmico

Quando todos os vértices de uma componente conexa apresentam a mesma temperatura, as diferenças de temperatura entre vértices conectados são nulas.

Considere, portanto,

$$u_1(t) = u_2(t) = \cdots = u_n(t).$$

Nesse caso,

$$Lu(t) = 0.$$

Consequentemente,

$$\frac{du(t)}{dt} = 0.$$

Assim, a distribuição de temperatura permanece constante ao longo do tempo, representando um estado de equilíbrio térmico da rede.

Esse resultado mostra que a matriz Laplaciana não apenas representa as conexões do grafo, mas também permite identificar matematicamente uma situação na qual não existem diferenças de temperatura capazes de produzir variação no sistema.

2.7.4 Solução do sistema de difusão

A equação diferencial

$$\frac{du(t)}{dt} = -cLu(t)$$

também permite representar a evolução das temperaturas a partir de uma distribuição inicial.

A solução formal do sistema pode ser escrita como

$$u(t) = e^{-cLt}u(0),$$

onde $u(0)$ representa a distribuição inicial de temperaturas.

Essa expressão mostra que a evolução da temperatura depende tanto da condição inicial quanto da estrutura de conexões representada pela matriz Laplaciana.

Dessa forma, o modelo estabelece uma relação direta entre o grafo e a dinâmica térmica: as arestas determinam quais vértices podem interagir, enquanto a matriz Laplaciana reúne essas informações e permite descrever matematicamente a evolução das temperaturas ao longo do tempo.

2.8 Interpretação da matriz Laplaciana

Depois de utilizar a matriz Laplaciana na construção do modelo de difusão de calor, é possível compreender melhor como seus elementos estão relacionados às diferenças de temperatura entre os vértices.

Considerando

$$L = D - A,$$

a i-ésima componente do produto Lu pode ser escrita como

$$(Lu)_i = d_i u_i - \sum_{j \sim i} u_j.$$

Como d_i representa o número de vizinhos do vértice v_i, essa expressão pode ser reorganizada na forma

$$(Lu)_i = \sum_{j \sim i} (u_i - u_j).$$

Portanto, a matriz Laplaciana compara a temperatura de cada vértice com as temperaturas dos vértices diretamente conectados a ele.

Se as temperaturas dos vértices vizinhos forem próximas, essas diferenças serão pequenas. Caso exista uma diferença significativa entre as temperaturas, a componente correspondente de Lu poderá apresentar maior módulo.

Essa interpretação é especialmente importante para o modelo estudado, pois a difusão de calor depende justamente das diferenças de temperatura entre vértices conectados.

2.8.1 Conservação e equilíbrio

Uma consequência da propriedade

$$L\mathbf{1} = 0$$

é que um vetor constante não produz variação quando submetido à dinâmica associada à matriz Laplaciana.

No caso do modelo de difusão de calor, se

$$u(t) = \begin{bmatrix} c \\ c \\ \vdots \\ c \end{bmatrix},$$

então

$$Lu(t) = 0.$$

Consequentemente,

$$\frac{du(t)}{dt} = 0.$$

Isso representa matematicamente o estado de equilíbrio térmico no qual todos os vértices possuem a mesma temperatura.

Além disso, como cada linha da matriz Laplaciana possui soma igual a zero, temos

$$\mathbf{1}^T L = 0.$$

Para a dinâmica

$$\frac{du(t)}{dt} = -cLu(t),$$

segue que

$$\frac{d}{dt}\left(\mathbf{1}^T u(t)\right) = -c\mathbf{1}^T L u(t) = 0.$$

Logo,

$$\mathbf{1}^T u(t) = \mathbf{1}^T u(0).$$

Isso significa que, no modelo de difusão considerado, a soma dos valores associados aos vértices permanece constante ao longo do tempo. No caso da temperatura, essa propriedade corresponde ao modelo idealizado em que não existem fontes externas, perdas ou ganhos de calor fora da rede.

2.8.2 Forma quadrática e energia da rede

Outra maneira de interpretar a matriz Laplaciana é por meio da forma quadrática

$$x^T L x = \sum_{\{i,j\}\in E} (x_i - x_j)^2.$$

Essa expressão permite medir o grau de diferença entre os valores associados a vértices conectados. Quando existem valores muito diferentes ligados por arestas, os termos $(x_i - x_j)^2$ aumentam. Quando os valores dos vértices conectados são semelhantes, esses termos diminuem.

Como cada termo da soma é não negativo,

$$x^T L x \geq 0.$$

Portanto, a matriz Laplaciana é semidefinida positiva para grafos simples não direcionados.

Além disso,

$$x^T L x = 0$$

quando os valores associados aos extremos de todas as arestas são iguais. Em um grafo conexo, isso ocorre somente quando x é um vetor constante.

Essa interpretação fornece uma relação entre a estrutura do grafo e a tendência de um processo de difusão reduzir diferenças entre vértices. Quanto maiores forem as diferenças existentes ao longo das arestas, maior será o valor da forma quadrática.

2.9 Exemplo da matriz Laplaciana

Considere a matriz de adjacência

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

Os graus dos vértices são

$$d_1 = 2, \qquad d_2 = 2, \qquad d_3 = 3, \qquad d_4 = 1.$$

Portanto,

$$D = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Aplicando

$$L = D - A,$$

obtemos

$$L = \begin{bmatrix} 2 & -1 & -1 & 0 \\ -1 & 2 & -1 & 0 \\ -1 & -1 & 3 & -1 \\ 0 & 0 & -1 & 1 \end{bmatrix}.$$

Esse exemplo mostra como a estrutura de conexões do grafo pode ser transformada em uma matriz que é utilizada na modelagem da difusão de calor. Os elementos da diagonal estão relacionados aos graus dos vértices, enquanto os elementos fora da diagonal representam as conexões entre eles.

2.10 Relação com a conectividade

Depois de compreender a utilização da matriz Laplaciana na difusão de calor, é possível analisar outras informações estruturais do grafo a partir dos autovalores dessa matriz.

Em particular, o número de autovalores nulos da matriz Laplaciana é igual ao número de componentes conexas do grafo. Assim, para um grafo conexo, existe exatamente um autovalor nulo.

Se os autovalores forem ordenados como

$$0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n,$$

então, em um grafo conexo,

$$\lambda_2 > 0.$$

O autovalor λ_2, conhecido como conectividade algébrica, fornece informações sobre a estrutura da rede. Sua análise também pode ser relacionada ao comportamento de processos dinâmicos no grafo, como a difusão de calor estudada anteriormente.

Dessa forma, a análise espectral é apresentada somente após a construção e interpretação do modelo de difusão, permitindo relacionar as propriedades estruturais do grafo com o comportamento do processo dinâmico.

3 Implementação computacional e Algoritmos em grafos

A implementação computacional desenvolvida neste trabalho utiliza a linguagem Python. O objetivo desta etapa é construir uma estrutura computacional capaz de representar grafos, realizar operações básicas sobre seus elementos, implementar algoritmos fundamentais de exploração e gerar diferentes estruturas topológicas para os experimentos.

A representação escolhida utiliza uma lista de adjacência, armazenada por meio de um dicionário da linguagem Python. Essa escolha permite associar cada vértice diretamente aos seus vizinhos, fornecendo uma estrutura adequada para a implementação dos algoritmos de busca e das demais operações realizadas neste trabalho.

A implementação considera, nesta etapa, grafos simples e não direcionados. Dessa forma, uma aresta entre os vértices u e v é representada simultaneamente como uma conexão de u para v e de v para u. Além disso, não são permitidos laços, isto é, arestas que conectem um vértice a ele mesmo, nem arestas duplicadas.

3.1 Estrutura de dados para grafos

A estrutura principal utilizada no projeto é a classe `Grafo`. Internamente, cada vértice é associado a uma lista contendo seus vizinhos.

A estrutura pode ser representada conceitualmente por

$$G = \{v_1 : N(v_1), v_2 : N(v_2), \ldots, v_n : N(v_n)\},$$

onde $N(v)$ representa o conjunto de vizinhos do vértice v.

A implementação desenvolvida é apresentada a seguir:

O método `adicionar_vertice()` insere um novo vértice caso ele ainda não exista na estrutura. O método `adicionar_aresta()` cria uma conexão entre dois vértices e, como o grafo é não direcionado, realiza a inserção nos dois sentidos.

Antes da criação da aresta, o método verifica se $u = v$. Caso isso ocorra, uma exceção é gerada, pois a implementação considera apenas grafos simples, nos quais não existem laços.

A implementação também verifica se a aresta já está presente na lista de adjacência. Dessa maneira, uma mesma aresta não pode ser inserida mais de uma vez.

Também foram implementadas operações de remoção de vértices e arestas, consulta de existência, obtenção dos vizinhos e cálculo do grau de um vértice.

O método `vizinhos()` retorna uma cópia da lista de vizinhos. Essa escolha impede que código externo altere diretamente a estrutura interna de adjacência do objeto.

Para um grafo não direcionado, vale a relação

$$\sum_{v \in V} \deg(v) = 2|E|,$$

pois cada aresta contribui com uma unidade para o grau de cada uma de suas duas extremidades.

Consequentemente, a quantidade de arestas pode ser obtida por

$$|E| = \frac{1}{2} \sum_{v \in V} \deg(v).$$

Na implementação, essa relação é utilizada pelo método `quantidade_arestas()`.

Essa representação apresenta como vantagem a possibilidade de acessar diretamente os vizinhos de cada vértice. Para grafos esparsos, nos quais o número de arestas é relativamente pequeno em comparação com o número máximo de conexões possíveis, a lista de adjacência é especialmente adequada.

3.2 Busca em largura – BFS

A Busca em Largura, conhecida pela sigla BFS (*Breadth-First Search*), é um algoritmo utilizado para percorrer ou explorar um grafo a partir de um vértice inicial.

Sua principal característica é explorar os vértices por níveis. Primeiramente, o algoritmo visita o vértice inicial. Em seguida, visita seus vizinhos diretos e, posteriormente, os vizinhos dos vértices encontrados anteriormente.

Assim, a exploração ocorre de acordo com a distância em número de arestas em relação ao vértice inicial.

Para controlar essa ordem, a BFS utiliza uma fila (*queue*), seguindo o princípio FIFO (*First In, First Out*), segundo o qual o primeiro elemento inserido é o primeiro a ser removido.

O procedimento pode ser resumido da seguinte maneira:

1. escolher um vértice inicial;
2. inserir o vértice inicial na fila;
3. marcá-lo como visitado;
4. retirar o primeiro vértice da fila;
5. analisar seus vizinhos;
6. inserir na fila os vizinhos ainda não visitados;
7. registrar a distância e o predecessor dos novos vértices;
8. repetir o processo enquanto existirem vértices na fila.

A marcação antecipada dos vértices como visitados evita que o mesmo vértice seja inserido diversas vezes na fila. Essa característica é importante principalmente em grafos que possuem ciclos.

Na implementação desenvolvida, foi utilizada a estrutura `deque`, disponibilizada pelo módulo `collections`. A operação `popleft()` permite remover o primeiro elemento da fila de forma eficiente.

A variável `fila` armazena os vértices que aguardam processamento. O conjunto `visitados` registra os vértices já descobertos. A lista `ordem` armazena a ordem em que os vértices são processados.

Além dessas informações, a implementação registra duas estruturas adicionais.

A primeira é `distancia`, que armazena a distância, em número de arestas, entre o vértice inicial e cada vértice alcançado. Para o vértice inicial, tem-se

$$d(s) = 0.$$

Quando um novo vértice v é descoberto a partir de um vértice u, sua distância é calculada por

$$d(v) = d(u) + 1.$$

A segunda estrutura é `predecessor`, que registra o vértice a partir do qual cada novo vértice foi descoberto. Dessa forma, é possível reconstruir a árvore de busca produzida pela BFS.

Por exemplo, se o vértice 2 for descoberto a partir do vértice 1, então

$$\text{predecessor}(2) = 1.$$

A partir dos predecessores, também é possível reconstruir um caminho entre dois vértices. Na implementação, essa funcionalidade é disponibilizada pelo método `caminho_bfs()`.

O trecho computacional correspondente encontra-se no Apêndice, na seção de códigos da busca em largura.

Em grafos não ponderados, a BFS encontra caminhos com o menor número de arestas a partir do vértice inicial. Portanto, o método `caminho_bfs()` pode ser utilizado para reconstruir um desses caminhos mínimos quando o destino é alcançável.

Além disso, a BFS pode ser utilizada para:

- verificar quais vértices são alcançáveis;
- verificar a conectividade de um grafo;
- calcular distâncias em número de arestas;
- construir uma árvore de busca;
- reconstruir caminhos;
- identificar componentes conexas;
- explorar estruturas de redes.

Utilizando uma lista de adjacência, cada vértice é processado no máximo uma vez e as arestas são analisadas durante a exploração.

Assim, a complexidade temporal da BFS é

$$O(|V| + |E|).$$

A complexidade espacial adicional é

$$O(|V|),$$

considerando o conjunto de vértices visitados, a fila, as distâncias, os predecessores e a ordem de visita.

3.3 Busca em profundidade – DFS

A Busca em Profundidade, conhecida pela sigla DFS (*Depth-First Search*), é outro algoritmo fundamental para exploração de grafos.

Diferentemente da BFS, que explora os vértices por níveis, a DFS procura avançar o máximo possível por um caminho antes de retornar para explorar outras possibilidades.

A estratégia pode ser descrita como um processo de aprofundamento seguido de retrocesso.

A partir de um vértice inicial, o algoritmo:

1. visita o vértice atual;
2. marca o vértice como visitado;
3. seleciona um vizinho ainda não visitado;
4. continua a busca a partir desse vizinho;
5. quando não existem novos vizinhos, retorna ao vértice anterior;
6. continua até que todos os vértices alcançáveis tenham sido visitados.

A DFS pode ser implementada utilizando uma pilha explícita ou por meio de chamadas recursivas. Na implementação desenvolvida neste trabalho, optou-se pela versão recursiva.

A implementação utilizada é:

O código correspondente encontra-se no Apêndice, na seção de algoritmos de busca.

A função interna `visitar()` realiza a exploração recursiva dos vértices.

Inicialmente, o vértice de partida é marcado como visitado. Em seguida, seus vizinhos são analisados. Quando um vizinho ainda não foi visitado, uma nova chamada da função é realizada utilizando esse vértice como ponto de partida.

O conjunto `visitados` evita que ciclos provoquem uma exploração indefinida.

Além da ordem de visita, a implementação registra o predecessor de cada vértice descoberto. Assim, a estrutura resultante permite representar a árvore de busca produzida pela DFS.

Por exemplo, considere o ciclo

$$0 \to 1 \to 2 \to 0.$$

Quando a DFS estiver processando o vértice 2, o vértice 0 já estará presente no conjunto de visitados. Portanto, a busca não realizará uma nova exploração a partir de 0.

A ordem produzida pela DFS depende da ordem em que os vizinhos aparecem na lista de adjacência. Dessa maneira, diferentes ordenações dos vizinhos podem produzir diferentes ordens de visita, mesmo quando o grafo analisado é o mesmo.

A DFS possui diversas aplicações, entre elas:

- exploração de grafos;
- identificação de componentes conexas;

- detecção de ciclos;
- construção de árvores de busca;
- análise de conectividade;
- análise estrutural de redes;
- resolução de problemas relacionados a caminhos;
- ordenação topológica em grafos direcionados.

Assim como na BFS, utilizando uma lista de adjacência, a complexidade temporal da DFS é

$$O(|V| + |E|).$$

Na implementação recursiva, a profundidade da pilha de chamadas pode atingir $O(|V|)$ no pior caso.

3.4 Conectividade e componentes conexas

A partir da BFS, foram implementados métodos para verificar a conectividade do grafo e identificar suas componentes conexas.

Um grafo não direcionado é conexo quando existe um caminho entre qualquer par de vértices. Equivalentemente, partindo de qualquer vértice, todos os demais vértices podem ser alcançados.

A implementação utiliza uma BFS a partir de um vértice arbitrário:

A implementação correspondente encontra-se no Apêndice, na seção de conectividade.

Se a quantidade de vértices alcançados pela BFS for igual à quantidade total de vértices do grafo, então o grafo é conexo.

Caso contrário, existem vértices que não podem ser alcançados a partir do vértice inicial e, portanto, o grafo não é conexo.

Além da verificação de conectividade, foi implementado o método `componentes_conexas()`, responsável por identificar os subgrafos conexos máximos do grafo.

A implementação correspondente encontra-se no Apêndice, na seção de conectividade e componentes conexas.

O algoritmo percorre os vértices do grafo e, sempre que encontra um vértice ainda não visitado, inicia uma nova BFS. Todos os vértices alcançados nessa busca pertencem à mesma componente conexa.

Dessa forma, a estrutura retornada pode ser representada conceitualmente por

$$\mathcal{C} = \{C_1, C_2, \ldots, C_k\},$$

onde cada C_i representa uma componente conexa do grafo.

3.5 Comparação entre BFS e DFS

Embora BFS e DFS possam ser utilizadas para percorrer os mesmos grafos, suas estratégias de exploração são diferentes.

A BFS utiliza uma fila e explora os vértices de acordo com seus níveis de distância a partir da origem. A DFS, por outro lado, procura aprofundar a exploração antes de retornar.

A Tabela 1 apresenta uma comparação entre os dois algoritmos.

Tabela 1: Comparação entre BFS e DFS.

Característica	**BFS**	**DFS**
Estrutura principal	Fila	Pilha ou recursão
Estratégia	Exploração por níveis	Exploração em profundidade
Distâncias em grafos não ponderados	Calcula distâncias mínimas	Não calcula diretamente
Caminho mínimo em grafo não ponderado	Pode encontrar	Não garante diretamente
Complexidade temporal	$O(\|V\| + \|E\|)$	$O(\|V\| + \|E\|)$
Controle de visitados	Conjunto de vértices visitados	Conjunto de vértices visitados
Aplicações	Distâncias, caminhos e conectividade	Ciclos, componentes e estrutura do grafo

No contexto deste trabalho, BFS e DFS constituem os primeiros algoritmos utilizados para explorar as estruturas geradas computacionalmente. A BFS também fornece informações adicionais de distância e predecessores, utilizadas posteriormente na reconstrução de caminhos.

3.6 Geração de ciclos

Além dos algoritmos de busca, foram implementadas funções para geração de diferentes classes de grafos. A primeira delas permite construir um grafo ciclo C_n.

Na implementação adotada, um ciclo simples deve possuir pelo menos três vértices. Cada vértice possui grau 2 e o último vértice está conectado novamente ao primeiro.

Antes da construção, o número de vértices é validado pelo método auxiliar `validar_n()`.

A função utilizada é:

A função correspondente encontra-se no Apêndice, na seção de geração de grafos.

O operador módulo é utilizado para realizar a conexão entre o último vértice e o primeiro. Por exemplo, para $n = 6$, a última conexão será

$$5 \to 0.$$

A Figura 2 apresenta o resultado da geração computacional do ciclo C_6.

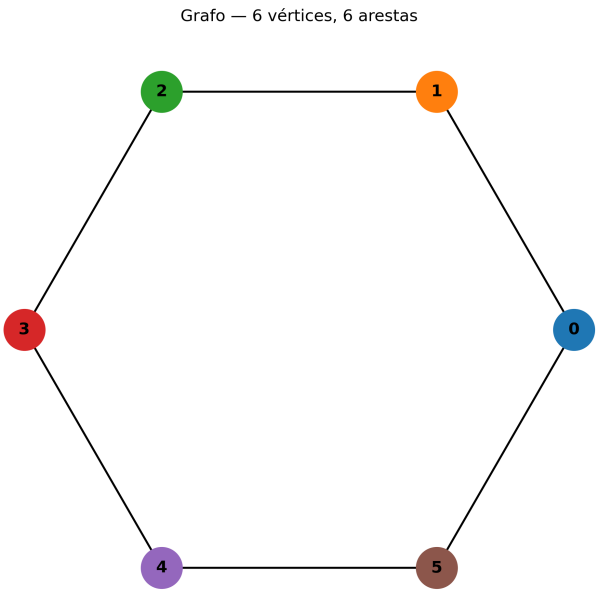


Figura 2: Grafo ciclo C_6 gerado computacionalmente.

3.7 Geração de grafos completos

Um grafo completo K_n é um grafo no qual cada par de vértices distintos está conectado por uma aresta.

Assim, cada vértice possui grau

$$\deg(v) = n - 1.$$

O número total de arestas é

$$|E| = \frac{n(n-1)}{2}.$$

A implementação utilizada é:

A implementação correspondente encontra-se no Apêndice, na seção de geração de grafos.

O limite

$$j = i + 1$$

faz com que cada par de vértices seja analisado apenas uma vez.

Para K_5, por exemplo,

$$|E| = \frac{5(5-1)}{2} = 10.$$

A Figura 3 apresenta o grafo completo K_5 gerado pelo programa.

3.8 Geração de grafos bipartidos

Um grafo bipartido é um grafo cujos vértices podem ser divididos em dois conjuntos disjuntos, de modo que as arestas conectem somente vértices pertencentes a conjuntos diferentes.

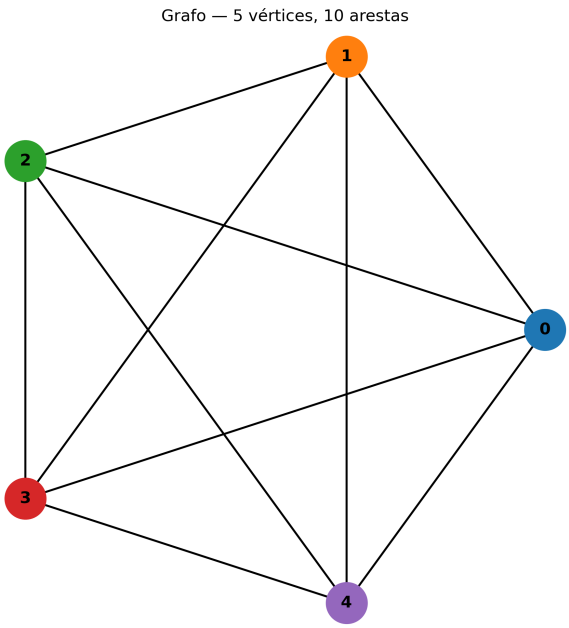


Figura 3: Grafo completo K_5 gerado computacionalmente.

Assim,

$$V = A \cup B$$

e

$$A \cap B = \varnothing.$$

Não existem, portanto, arestas entre dois vértices pertencentes simultaneamente a A ou simultaneamente a B.

Um caso particular é o grafo bipartido completo $K_{n,m}$, no qual cada vértice do conjunto A está conectado a todos os vértices do conjunto B.

O número de arestas é

$$|E| = nm.$$

A implementação utilizada é:

A implementação correspondente encontra-se no Apêndice, na seção de geração de grafos.

Nesse procedimento, os vértices do primeiro conjunto são numerados de 0 até $n-1$, enquanto os vértices do segundo conjunto são numerados de n até $n+m-1$.

Como exemplo, foi construído o grafo bipartido completo $K_{3,2}$. Nesse caso,

$$|A| = 3$$

e

$$|B| = 2.$$

Consequentemente,

$$|E| = 3 \cdot 2 = 6.$$

A Figura 4 apresenta o resultado da geração computacional do grafo $K_{3,2}$.

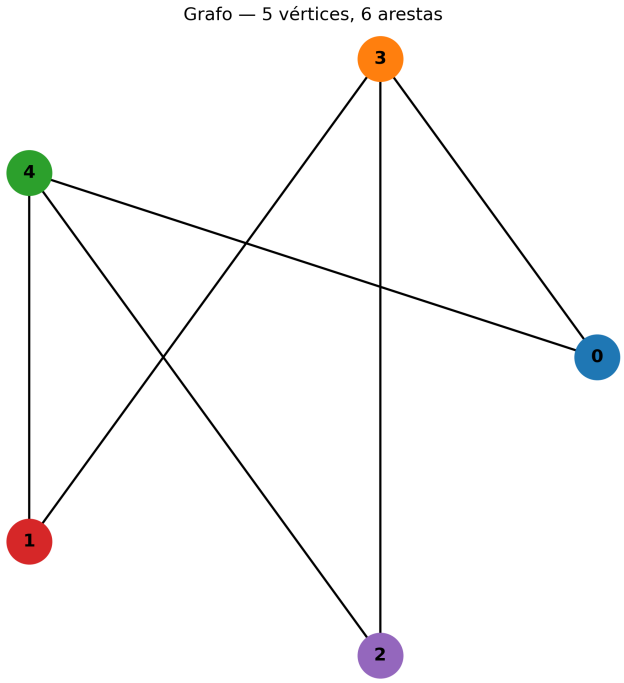


Figura 4: Grafo bipartido completo $K_{3,2}$ gerado computacionalmente.

A geração dessa classe de grafos permite estudar estruturas nas quais os elementos podem ser naturalmente divididos em dois grupos.

3.9 Geração de grafos aleatórios

Para permitir a realização de experimentos com diferentes estruturas de rede, também foi implementada uma função para geração de grafos aleatórios.

A construção utiliza o modelo de Erdős–Rényi, no qual cada possível aresta entre dois vértices é criada de acordo com uma probabilidade p.

A função recebe o número de vértices, a probabilidade de conexão e, opcionalmente, uma semente para reprodução do experimento.

A implementação utilizada é:

A implementação correspondente encontra-se no Apêndice, na seção de geração de grafos.

O parâmetro n representa o número de vértices e p representa a probabilidade de criação de cada possível aresta. No máximo

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

arestas serão adicionadas (caso $G = K_n$ um grafo completo) em um grafo simples não direcionado com n vértices.

Quando

$$p = 0,$$

nenhuma aresta é adicionada.

Quando

$$p = 1,$$

todas as possíveis arestas são adicionadas, resultando em um grafo completo.
Para

$$0 < p < 1,$$

a quantidade de arestas varia de acordo com o processo aleatório.
O número esperado de arestas é

$$\mathbb{E}[|E|] = p\frac{n(n-1)}{2}.$$

A implementação utiliza um objeto local da classe `random.Random`. Dessa forma, a utilização de uma semente não altera o estado global do gerador aleatório utilizado por outras partes do programa.

Por exemplo:

O exemplo de chamada correspondente encontra-se no Apêndice, na seção de geração de grafos.

Nesse caso, são utilizados oito vértices e probabilidade 0.3 para cada possível conexão.

A utilização de sementes é importante em experimentos computacionais, pois permite reproduzir uma mesma estrutura de entrada em diferentes execuções. Isso possibilita comparar algoritmos utilizando exatamente a mesma rede.

3.10 Geração paramétrica de grafos

Também foi implementada uma função auxiliar que permite selecionar determinados tipos de grafos por meio de um parâmetro textual.

A função atualmente contempla os tipos ciclo e completo:

A função correspondente encontra-se no Apêndice, na seção de geração de grafos.

Essa estrutura permite centralizar a seleção do tipo de grafo e facilita a expansão futura do programa para outras classes de estruturas.

3.11 Visualização dos grafos

A visualização dos grafos foi implementada utilizando a biblioteca `matplotlib`. O objetivo dessa etapa é permitir observar graficamente a estrutura das redes geradas e verificar visualmente as conexões entre os vértices.

Para facilitar a interpretação, os vértices são distribuídos em uma circunferência. Considerando um grafo com n vértices, a posição angular de cada vértice pode ser definida por

$$\theta_i = \frac{2\pi i}{n}.$$

As coordenadas cartesianas são então calculadas por

$$x_i = \cos(\theta_i)$$

e

$$y_i = \sin(\theta_i).$$

Dessa maneira, todos os vértices são posicionados sobre uma circunferência.

Na implementação, esse cálculo é realizado diretamente dentro da função `visualizar_grafo()`:

O trecho correspondente encontra-se no Apêndice, na seção de visualização.

As arestas são então desenhadas entre as coordenadas dos respectivos vértices.

Como o grafo é não direcionado, cada aresta aparece duas vezes na lista de adjacência. Para evitar sua representação duplicada na figura, utiliza-se um conjunto de arestas já desenhadas:

O trecho correspondente encontra-se no Apêndice, na seção de visualização.

A utilização de `frozenset` permite representar o par $\{u, v\}$ independentemente da ordem em que os vértices aparecem. Assim, as representações (u, v) e (v, u) são consideradas a mesma aresta.

Os vértices são representados graficamente por pontos e identificados por seus respectivos rótulos.

A visualização permite observar diferentes estruturas geradas pelo programa, como ciclos, grafos completos, grafos bipartidos e grafos aleatórios.

A utilização de uma mesma convenção visual também facilita a comparação entre diferentes redes.

3.12 Salvamento das figuras

Além da visualização na tela, as figuras produzidas durante os experimentos podem ser salvas automaticamente no diretório do programa.

Para obter o diretório no qual o arquivo Python está localizado, utiliza-se:

O código correspondente encontra-se no Apêndice, na seção de salvamento das figuras.

Essa abordagem permite construir o caminho da imagem a partir da localização do próprio programa.

Para aumentar a qualidade das figuras utilizadas no relatório, o salvamento utiliza resolução de 300 pontos por polegada:

O código correspondente encontra-se no Apêndice, na seção de salvamento das figuras.

As figuras utilizadas ao longo deste relatório foram produzidas a partir das estruturas geradas computacionalmente.

3.13 Síntese da implementação

A implementação desenvolvida nesta etapa estabelece a base computacional necessária para as etapas posteriores do projeto.

Foram implementadas operações fundamentais para representação e manipulação de grafos, incluindo:

- criação de vértices;
- remoção de vértices;
- criação e remoção de arestas;

- consulta de existência de vértices e arestas;
- obtenção dos vizinhos de um vértice;
- cálculo do grau dos vértices;
- cálculo da quantidade de vértices e arestas;
- Busca em Largura (BFS);
- cálculo de distâncias pela BFS;
- registro de predecessores;
- reconstrução de caminhos mínimos em grafos não ponderados;
- Busca em Profundidade (DFS);
- verificação de conectividade;
- identificação de componentes conexas;
- geração de ciclos;
- geração de grafos completos;
- geração de grafos bipartidos;
- geração de grafos aleatórios;
- geração paramétrica de determinados tipos de grafos;
- visualização das estruturas;
- salvamento das figuras geradas.

Essas funcionalidades permitem construir diferentes tipos de redes e analisá-las por meio de algoritmos de exploração. A estrutura baseada em lista de adjacência fornece a representação necessária para as etapas seguintes do projeto, especialmente para os estudos relacionados à matriz de adjacência, matriz de graus e matriz Laplaciana.

A implementação atual considera grafos simples e não direcionados. Essa definição estabelece a base para a incorporação de estruturas adicionais nas próximas etapas.

4 Matrizes associadas aos grafos

Nesta etapa, a representação por listas de adjacência é utilizada como estrutura de origem para a construção das matrizes associadas ao grafo. Seja $G = (V, E)$ um grafo simples e não direcionado, com $V = \{v_1, \ldots, v_n\}$. Para que as linhas e colunas das matrizes correspondam consistentemente aos vértices, fixa-se uma ordenação $\varphi : V \to \{1, \ldots, n\}$. Essa correspondência é mantida durante toda a construção, evitando que a posição de um vértice varie entre as matrizes.

A conversão para a representação matricial permite utilizar operações de álgebra linear na análise da estrutura do grafo. Em particular, a matriz Laplaciana estabelece a ligação entre a topologia da rede e os modelos dinâmicos estudados nas seções posteriores.

4.1 Matriz de adjacência

A matriz de adjacência é construída diretamente a partir das listas de vizinhos armazenadas pela estrutura do grafo. Inicializa-se uma matriz $A \in \mathbb{R}^{n\times n}$ com entradas nulas. Para cada vértice v_i e cada vizinho v_j identificado em sua lista de adjacência, atribui-se

$$A_{ij} = 1.$$

Como o grafo considerado é simples e não direcionado, não há laços nem arestas múltiplas. Consequentemente,

$$A_{ii} = 0 \quad \text{e} \quad A_{ij} = A_{ji}, \qquad 1 \leq i, j \leq n.$$

A simetria decorre do fato de que, se $\{v_i, v_j\} \in E$, então cada um dos dois vértices aparece na lista de adjacência do outro.

A construção pode ser expressa por

$$A_{ij} = \begin{cases} 1, & \text{se } \{v_i, v_j\} \in E, \\ 0, & \text{caso contrário.} \end{cases}$$

Na implementação, a lista de adjacência é percorrida uma vez para preencher as posições correspondentes aos pares de vértices adjacentes. A ordenação φ é utilizada para converter os identificadores dos vértices em índices da matriz.

Como verificação da consistência entre as duas representações, para todo $v_i \in V$, o número de entradas não nulas na linha i deve coincidir com o número de vizinhos armazenados para esse vértice. Além disso, por se tratar de um grafo não direcionado, a matriz obtida deve ser simétrica. A soma de todas as entradas fornece o dobro do número de arestas:

$$\sum_{i=1}^{n}\sum_{j=1}^{n} A_{ij} = 2|E|,$$

pois cada aresta é contabilizada nas posições (i, j) e (j, i).

4.2 Matriz de graus

A matriz de graus é obtida a partir dos graus já calculados pela estrutura $Grafo$. Para cada vértice v_i, seja $d(v_i)$ a quantidade de seus vizinhos. Constrói-se a matriz diagonal $D \in \mathbb{R}^{n\times n}$ por

$$D_{ij} = \begin{cases} d(v_i), & i = j, \\ 0, & i \neq j. \end{cases}$$

Assim, os elementos da diagonal principal armazenam os graus dos vértices, enquanto as demais posições permanecem nulas.

A compatibilidade entre as matrizes de adjacência e de graus é verificada pela identidade

$$d(v_i) = \sum_{j=1}^{n} A_{ij} = D_{ii}.$$

Também se verifica que

$$\operatorname{tr}(D) = \sum_{i=1}^{n} d(v_i) = 2|E|,$$

que corresponde ao lema do aperto de mãos para grafos não direcionados. Essas igualdades permitem detectar inconsistências entre as listas de adjacência, os graus armazenados e as matrizes construídas.

4.3 Matriz Laplaciana e propriedades

A matriz Laplaciana é construída a partir das matrizes de graus e de adjacência, por meio da operação

$$L = D - A.$$

Portanto, seus elementos são dados por

$$L_{ij} = \begin{cases} d(v_i), & i = j, \\ -1, & i \neq j \text{ e } \{v_i, v_j\} \in E, \\ 0, & \text{caso contrário.} \end{cases}$$

A construção utiliza as mesmas correspondências entre vértices e índices adotadas nas duas matrizes anteriores. Dessa forma, cada linha e coluna de L representa o mesmo vértice nas três estruturas.

A implementação é validada por propriedades que decorrem da definição da Laplaciana. Como A e D são simétricas, L também deve ser simétrica:

$$L = L^{\mathsf{T}}.$$

Além disso, a soma dos elementos de cada linha é nula:

$$L\mathbf{1} = \mathbf{0},$$

em que $\mathbf{1} \in \mathbb{R}^n$ é o vetor cujas componentes são todas iguais a um. Essa propriedade decorre da igualdade entre o grau de um vértice e a quantidade de entradas unitárias na linha correspondente de A.

Para qualquer vetor $x \in \mathbb{R}^n$, vale ainda

$$x^{\mathsf{T}} L x = \sum_{\{v_i, v_j\} \in E} (x_i - x_j)^2 \geq 0.$$

Logo, L é semidefinida positiva e todos os seus autovalores são não negativos. O número de componentes conexas do grafo coincide com a multiplicidade do autovalor nulo. Em particular, se G é conexo, o núcleo de L é gerado pelo vetor $\mathbf{1}$, e o autovalor zero tem multiplicidade um.

Essas verificações são importantes para assegurar que a matriz utilizada nas etapas de análise espectral e de modelagem da difusão representa corretamente a estrutura da rede. No modelo dinâmico considerado posteriormente, a Laplaciana aparece na equação

$$\dot{u}(t) = -cLu(t),$$

de modo que um erro na construção de L alteraria diretamente as taxas de troca entre vértices.

5 Árvores geradoras de peso mínimo

Para tratar problemas em que as conexões possuem custos distintos, a estrutura de grafo é estendida para armazenar pesos associados às arestas. Essa extensão permite distinguir a existência de uma conexão de seu custo e fornece os dados necessários à execução do algoritmo de Kruskal.

5.1 Grafos ponderados

Na representação não ponderada utilizada anteriormente, a lista de adjacência registra apenas quais vértices são vizinhos. Na representação ponderada, cada conexão passa a ser acompanhada de um valor real $w(e)$, denominado peso da aresta. Assim, para uma aresta $e = \{u, v\}$, registra-se tanto o vértice adjacente quanto o peso correspondente.

Formalmente, considera-se uma função de pesos

$$w : E \to \mathbb{R}.$$

Para manter a natureza não direcionada do grafo, o peso associado à conexão $\{u, v\}$ é o mesmo independentemente do sentido em que a lista de adjacência é consultada. A estrutura deve, portanto, preservar a consistência entre os registros de u para v e de v para u.

Na preparação dos dados para Kruskal, cada aresta é representada uma única vez, por exemplo, por um par não ordenado $\{u, v\}$ acompanhado de seu peso. Essa condição evita que a mesma conexão seja inserida duas vezes na lista de arestas a ordenar. A lista resultante é organizada em ordem não decrescente dos pesos:

$$w(e_1) \leq w(e_2) \leq \cdots \leq w(e_{|E|}).$$

A ordenação constitui a etapa inicial do algoritmo de Kruskal.

5.2 Algoritmo de Kruskal e Union-Find

O algoritmo de Kruskal constrói uma árvore geradora mínima selecionando arestas em ordem não decrescente de peso e aceitando somente aquelas que conectam componentes distintas. Para realizar essa verificação de maneira eficiente, utiliza-se a estrutura de conjuntos disjuntos, conhecida como Union-Find ou Disjoint Set Union (DSU).

Inicialmente, cada vértice constitui uma componente isolada. A estrutura mantém, para cada elemento, um representante de seu conjunto. A operação find(v) retorna o representante da componente que contém v, enquanto union(u, v) combina os conjuntos que contêm u e v, caso sejam distintos.

Duas otimizações usuais tornam essas operações eficientes. A compressão de caminhos modifica os ponteiros percorridos durante uma operação find, fazendo com que os vértices consultados passem a apontar diretamente para o representante. A união por posto ou por tamanho anexa a árvore de menor altura estimada, ou o conjunto menor, à estrutura maior. Essas estratégias reduzem o custo amortizado das operações da DSU.

Após ordenar as arestas, o algoritmo percorre a sequência $e_1, \ldots, e_{|E|}$. Para cada aresta $e_k = \{u, v\}$, são consultados os representantes de suas extremidades. Se

$$\text{find}(u) \neq \text{find}(v),$$

a aresta é adicionada à solução e os conjuntos são unidos. Caso contrário, a aresta é descartada, pois sua inclusão produziria um ciclo no conjunto de arestas selecionadas.

Seja T o conjunto de arestas aceitas e W_T o peso acumulado. A cada inclusão, atualiza-se

$$T \leftarrow T \cup \{\{u, v\}\}, \qquad W_T \leftarrow W_T + w(\{u, v\}).$$

O procedimento termina quando são selecionadas $n - 1$ arestas ou quando todas as arestas disponíveis são examinadas. No primeiro caso, se o grafo possui n vértices, as arestas selecionadas formam uma árvore geradora. No segundo caso, se foram selecionadas menos de $n - 1$ arestas, o grafo é desconexo e não existe árvore geradora que contenha todos os seus vértices.

Quando G é conexo, o algoritmo produz uma árvore geradora mínima, isto é, uma árvore $T = (V, E_T)$ que minimiza

$$w(T) = \sum_{e \in E_T} w(e)$$

entre todas as árvores geradoras de G. A correção do procedimento decorre da propriedade do corte: em cada etapa, uma aresta de menor peso que conecta duas componentes distintas pode ser escolhida sem excluir a possibilidade de completar uma solução ótima.

Ao final da execução, são verificadas as condições

$$|E_T| = |V| - 1 \quad \text{e} \quad T \text{ é conexo.}$$

Também se confere se o peso total informado corresponde à soma dos pesos das arestas efetivamente selecionadas. Essas verificações permitem avaliar a consistência do resultado produzido pelo algoritmo e distinguir a obtenção de uma árvore geradora da situação em que o grafo de entrada não é conexo.

6 Teoria espectral de grafos

A matriz Laplaciana permite estudar propriedades estruturais de um grafo por meio de ferramentas da álgebra linear. Em particular, seus autovalores e autovetores fornecem informações sobre a conectividade da rede e sobre os modos de evolução de processos definidos em seus vértices. Nesta seção, aprofundam-se as propriedades espectrais da matriz Laplaciana introduzida anteriormente e apresenta-se sua análise numérica para a rede com duas comunidades utilizada nos experimentos.

Considere um grafo simples, não direcionado e não ponderado $G = (V, E)$, com $N = |V|$ vértices. Sejam A sua matriz de adjacência, D sua matriz diagonal de graus e

$$L = D - A$$

sua matriz Laplaciana combinatória. Como o grafo é não direcionado, $A = A^T$, e, consequentemente, $L = L^T$. Além disso, a matriz L é semidefinida positiva, isto é,

$$x^T L x \geq 0, \qquad \forall x \in \mathbb{R}^N.$$

Essas propriedades garantem que seus autovalores são reais e não negativos, permitindo ordená-los de maneira crescente.

6.1 Autovalores e autovetores

Um escalar λ é denominado autovalor de uma matriz quadrada L quando existe um vetor não nulo v tal que

$$Lv = \lambda v.$$

O vetor v é denominado autovetor associado ao autovalor λ. De forma equivalente, os autovalores são as raízes do polinômio característico

$$p_L(\lambda) = \det(L - \lambda I),$$

em que I representa a matriz identidade de ordem N.

Como a matriz Laplaciana de um grafo não direcionado é real e simétrica, o teorema espectral garante a existência de uma base ortonormal de autovetores. Assim, existem uma matriz ortogonal Q e uma matriz diagonal Λ tais que

$$L = Q\Lambda Q^T,$$

com

$$Q^TQ = QQ^T = I \quad \text{e} \quad \Lambda = \operatorname{diag}(\lambda_1, \lambda_2, \ldots, \lambda_N).$$

As colunas de Q são os autovetores normalizados de L, enquanto os elementos da diagonal de Λ são os respectivos autovalores.

Para a Laplaciana combinatória, todos os autovalores são não negativos. Adotando a ordenação crescente, escreve-se

$$0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_N.$$

A igualdade $\lambda_1 = 0$ decorre do fato de que a soma dos elementos de cada linha de L é nula. Portanto,

$$L\mathbf{1} = \mathbf{0},$$

em que $\mathbf{1}$ é o vetor cujas componentes são todas iguais a um. Logo, um autovetor associado ao autovalor nulo é

$$v_1 = \frac{1}{\sqrt{N}}\mathbf{1}.$$

Esse vetor representa um estado uniforme: todos os vértices possuem o mesmo valor associado.

A multiplicidade do autovalor nulo é igual ao número de componentes conexas do grafo. Consequentemente, se o grafo é conexo, o autovalor nulo é simples e vale

$$0 = \lambda_1 < \lambda_2.$$

Essa propriedade estabelece uma relação direta entre a estrutura de conectividade e o espectro da matriz Laplaciana.

Os demais autovetores descrevem padrões de variação entre os vértices. Em particular, como a matriz L é simétrica, autovetores associados a autovalores distintos são ortogonais. Assim, para $i \neq 1$,

$$v_i^T\mathbf{1} = 0,$$

o que significa que a soma das componentes de cada um desses autovetores é nula. Tais vetores representam modos não uniformes da rede.

Essa interpretação também pode ser relacionada à dinâmica de difusão apresentada anteriormente. Para o sistema

$$\frac{du(t)}{dt} = -cLu(t),$$

a decomposição espectral permite escrever a condição inicial como

$$u(0) = \sum_{i=1}^{N} \alpha_i v_i, \qquad \alpha_i = v_i^T u(0),$$

e, portanto, a solução é

$$u(t) = \sum_{i=1}^{N} \alpha_i e^{-c\lambda_i t} v_i.$$

O modo associado a $\lambda_1 = 0$ permanece constante, enquanto os modos associados aos autovalores positivos decaem exponencialmente. Dessa forma, os autovalores determinam as taxas de relaxação dos diferentes padrões de temperatura, e os autovetores indicam a distribuição espacial desses padrões na rede.

6.2 Espectro da matriz Laplaciana

O conjunto dos autovalores de uma matriz, considerados com suas multiplicidades algébricas, é denominado espectro. Para a matriz Laplaciana de um grafo simples não direcionado, o espectro é formado por N valores reais não negativos:

$$\mathrm{Spec}(L) = \{\lambda_1, \lambda_2, \ldots, \lambda_N\}.$$

A análise desse conjunto permite identificar propriedades que não são imediatamente evidentes apenas pela inspeção da lista de adjacência ou da matriz de adjacência.

No presente trabalho, a análise numérica foi realizada para a rede aleatória com duas comunidades descrita na seção experimental. A construção utiliza duas comunidades com dez vértices cada, de modo que

$$|A| = |B| = 10, \qquad |V| = 20.$$

As probabilidades de conexão são $p = 0{,}80$ para pares pertencentes à mesma comunidade e $q = 0{,}05$ para pares pertencentes a comunidades distintas. Foi utilizada a semente pseudoaleatória 42. Na realização obtida, a rede possui 76 arestas, sendo 36 internas à primeira comunidade, 37 internas à segunda e três entre comunidades.

A partir da matriz de adjacência A, constrói-se a matriz diagonal de graus D e calcula-se

$$L = D - A.$$

Em seguida, os autovalores e autovetores são obtidos por diagonalização numérica de uma matriz real simétrica. Para esse tipo de matriz, utiliza-se um procedimento específico para matrizes simétricas, como `numpy.linalg.eigh`, que retorna os autovalores em ordem crescente e os autovetores ortonormais correspondentes.

A diagonalização numérica da matriz 20×20, aplicada à realização reproduzida com a semente 42, fornece o seguinte espectro aproximado:

$$\begin{aligned}\mathrm{Spec}(L) \approx \{&0{,}000000,\ 0{,}471080,\ 4{,}696175,\ 5{,}632593,\\ &6{,}367695,\ 6{,}798695,\ 7{,}495950,\ 7{,}674356,\\ &7{,}770021,\ 7{,}979593,\ 8{,}000000,\ 8{,}416094,\\ &9{,}340908,\ 9{,}500075,\ 9{,}539704,\ 9{,}714269,\\ &9{,}762846,\ 9{,}972273,\ 11{,}147668,\ 11{,}720003\}.\end{aligned}$$

Os valores são apresentados com seis casas decimais e devem ser interpretados como aproximações numéricas. O primeiro autovalor é numericamente nulo, enquanto os demais são positivos. Portanto, há um único autovalor nulo, o que é consistente com a existência de uma única componente conexa na rede, também verificada pelo procedimento de busca em largura.

O espectro apresenta uma separação entre o segundo autovalor e os demais valores positivos. Essa característica é compatível com a organização em duas comunidades: existem muitas conexões no interior de cada grupo, mas apenas três arestas conectam os grupos entre si. Assim, a transferência de uma quantidade entre as comunidades depende de um conjunto relativamente pequeno de conexões, o que produz um modo de variação associado à comunicação entre os dois grupos.

Para ilustrar a interpretação dos autovetores, considere o autovetor v_2, associado a λ_2. Com uma escolha de sinal em que as componentes da primeira comunidade são predominantemente positivas, suas componentes aproximadas são:

$$\begin{aligned}v_2 \approx (&0{,}2330,\ 0{,}1932,\ 0{,}2309,\ 0{,}1411,\ 0{,}2486,\\ &0{,}2305,\ 0{,}2333,\ 0{,}2334,\ 0{,}2432,\ 0{,}2333,\\ &-0{,}1868,\ -0{,}2330,\ -0{,}2333,\ -0{,}2395,\ -0{,}1988,\\ &-0{,}2310,\ -0{,}1851,\ -0{,}2323,\ -0{,}2330,\ -0{,}2478)^T.\end{aligned}$$

As dez primeiras componentes correspondem aos vértices da primeira comunidade, e as dez últimas, aos vértices da segunda. Observa-se que as componentes apresentam sinais predominantemente opostos nos dois grupos. Isso indica que o modo associado ao segundo menor autovalor representa, principalmente, uma diferença entre as comunidades.

O sinal global de um autovetor não é único: se v é autovetor, então $-v$ também é autovetor associado ao mesmo autovalor. Portanto, a interpretação relevante não depende de qual comunidade aparece com sinal positivo, mas da oposição entre os sinais e da organização das componentes. Além disso, os valores apresentados são aproximações numéricas e não devem ser interpretados como uma classificação rígida dos vértices.

É importante destacar que o código original do laboratório constrói a matriz Laplaciana, mas não implementa uma rotina própria para calcular autovalores ou autovetores. Os valores apresentados nesta seção correspondem a um pós-processamento numérico da rede reproduzida com os parâmetros informados, e não a uma saída calculada automaticamente pela interface de simulação.

6.3 Segundo menor e maior autovalor

Entre os autovalores da matriz Laplaciana, o segundo menor e o maior possuem importância particular para o estudo da conectividade e da dinâmica de difusão. O

segundo menor autovalor, λ_2, é denominado conectividade algébrica. Para um grafo não direcionado, vale

$$\lambda_2 > 0 \quad \Longleftrightarrow \quad G \text{ é conexo.}$$

Assim, λ_2 fornece um critério espectral para verificar a conectividade, complementando os métodos de busca utilizados nas seções anteriores.

Na rede analisada, obteve-se

$$\boxed{\lambda_2 \approx 0{,}471080}.$$

Como esse valor é positivo, a matriz Laplaciana possui apenas um autovalor nulo, confirmando espectralmente que a rede é conexa. Entretanto, a conectividade algébrica também fornece informação sobre a intensidade da conexão global. Valores menores de λ_2 estão associados a redes que podem ser separadas em grupos com poucas conexões entre si. No caso estudado, a existência de apenas três arestas entre as duas comunidades é compatível com o valor relativamente pequeno de λ_2 em comparação com os demais autovalores positivos.

A relação com a dinâmica de difusão pode ser observada pela solução espectral

$$u(t) = \sum_{i=1}^{N} \alpha_i e^{-c\lambda_i t} v_i.$$

Desconsiderando o modo constante, que não decai, o modo associado a λ_2 é o mais lento a desaparecer, pois corresponde ao menor autovalor positivo. Sua taxa de decaimento é

$$c\lambda_2.$$

Para a rede considerada, isso resulta em

$$c\lambda_2 \approx \begin{cases} 0{,}004711, & c = 0{,}01, \\ 0{,}037686, & c = 0{,}08. \end{cases}$$

Esses valores representam as taxas exponenciais do modo mais lento não constante na solução contínua. Portanto, mantendo a rede e a condição inicial, o aumento de c acelera o decaimento dos modos não uniformes. Essa conclusão é coerente com a comparação visual dos experimentos, nos quais a configuração com $c = 0{,}08$ apresenta uma equalização térmica mais rápida que a configuração com $c = 0{,}01$. A comparação entre os vídeos, contudo, permanece qualitativa, pois não foram registrados históricos numéricos completos que permitam estimar tempos experimentais de convergência.

O maior autovalor também é relevante, especialmente para a estabilidade da discretização temporal. Para a rede analisada, obteve-se

$$\boxed{\lambda_N = \lambda_{20} \approx 11{,}720003}.$$

Esse valor determina o modo espectral de maior frequência e impõe uma restrição ao passo temporal quando se utiliza o método de Euler explícito. Aplicando esse método ao sistema

$$\frac{du(t)}{dt} = -cLu(t),$$

obtém-se a atualização

$$u^{k+1} = (I - c\Delta t L)u^k.$$

Se v_i é um autovetor de L associado a λ_i, o fator de amplificação do modo correspondente é

$$\mu_i = 1 - c\Delta t\lambda_i.$$

A estabilidade linear exige que

$$|\mu_i| \leq 1 \qquad \text{para todo } i.$$

Como $c > 0$, $\Delta t > 0$ e $0 \leq \lambda_i \leq \lambda_N$, essa condição equivale a

$$0 \leq c\Delta t\lambda_i \leq 2.$$

Para que os modos não constantes sejam estritamente amortecidos, sem permanecer no limite de oscilação de módulo unitário, utiliza-se a condição estrita

$$0 < c\Delta t\lambda_N < 2.$$

Consequentemente, uma condição suficiente para a escolha do passo temporal é

$$0 < \Delta t < \frac{2}{c\lambda_N}.$$

Nos experimentos apresentados, foi utilizado $\Delta t = 1{,}0$. Para $c = 0{,}01$, o produto associado ao maior autovalor é

$$c\Delta t\lambda_N \approx 0{,}01 \cdot 1{,}0 \cdot 11{,}720003 \approx 0{,}1172.$$

Para $c = 0{,}08$, obtém-se

$$c\Delta t\lambda_N \approx 0{,}08 \cdot 1{,}0 \cdot 11{,}720003 \approx 0{,}9376.$$

Ambos os valores pertencem ao intervalo $(0, 2)$. Portanto, os dois cenários satisfazem a condição de estabilidade do método de Euler explícito para a matriz Laplaciana considerada. Em particular, o maior valor de c utilizado nos vídeos não viola a restrição de estabilidade para $\Delta t = 1{,}0$.

A análise mostra que os dois extremos do espectro desempenham papéis complementares. O segundo menor autovalor está relacionado à conectividade algébrica e ao modo não constante de relaxação mais lenta; o maior autovalor determina a restrição mais exigente para a estabilidade do método explícito. Assim, a escolha do parâmetro de difusão e do passo temporal deve considerar simultaneamente λ_2 e λ_N: aumentar c acelera a difusão contínua, mas também aumenta o produto $c\Delta t\lambda_N$, reduzindo a margem de estabilidade numérica.

Por fim, ressalta-se que a condição de estabilidade depende da combinação entre a rede, o coeficiente de difusão e o passo temporal. Portanto, os valores utilizados nos experimentos não devem ser generalizados para outras redes sem recalcular ou estimar o maior autovalor da respectiva matriz Laplaciana.

7 Modelagem e discretização do sistema dinâmico

A dinâmica estudada descreve a redistribuição de uma quantidade escalar associada aos vértices de um grafo. Nos experimentos, essa quantidade é interpretada como temperatura, mas a formulação também pode representar outros processos de difusão em redes. Considere um grafo simples, não direcionado e não ponderado $G = (V, E)$, com $|V| = N$, matriz de adjacência A, matriz diagonal de graus D e matriz Laplaciana $L = D - A$.

7.1 Variáveis associadas aos vértices

A cada vértice $v_i \in V$ associa-se uma variável de estado $u_i(t)$, que representa a quantidade de interesse no instante t. Na interpretação térmica, $u_i(t)$ é a temperatura do vértice v_i. O estado completo da rede é representado por

$$u(t) = \begin{pmatrix} u_1(t) \\ u_2(t) \\ \vdots \\ u_N(t) \end{pmatrix} \in \mathbb{R}^N.$$

A ordenação das componentes segue a identificação dos vértices utilizada na implementação. As interações diretas entre componentes são determinadas pelas arestas: dois vértices só trocam estado diretamente quando são adjacentes.

7.2 Modelo de difusão

Seja $c > 0$ o coeficiente de difusão. Para cada vértice v_i, a taxa de variação do estado é proporcional à soma das diferenças entre os estados dos seus vizinhos e o seu próprio estado:

$$\frac{du_i(t)}{dt} = c \sum_{v_j \sim v_i} \big(u_j(t) - u_i(t)\big),$$

em que $v_j \sim v_i$ indica que existe uma aresta entre v_i e v_j. Assim, um vértice com valor superior ao de seus vizinhos tende a diminuir seu estado, enquanto um vértice com valor inferior tende a aumentá-lo.

Pela definição da Laplaciana,

$$(Lu(t))_i = d_i u_i(t) - \sum_{v_j \sim v_i} u_j(t),$$

onde d_i é o grau de v_i. Logo, o sistema pode ser escrito matricialmente como

$$\frac{du(t)}{dt} = -cLu(t).$$

Como $L\mathbf{1} = 0$, a soma dos estados é conservada:

$$\frac{d}{dt}\mathbf{1}^T u(t) = -c\mathbf{1}^T Lu(t) = 0.$$

Portanto, a média global $\overline{u}(t) = \frac{1}{N}\sum_{i=1}^{N} u_i(t)$ permanece constante. Se o grafo for conexo, o estado de equilíbrio é uniforme e igual à média inicial:

$$\lim_{t\to\infty} u(t) = \overline{u}(0)\mathbf{1}.$$

7.3 Solução contínua

A solução formal do sistema diferencial linear é

$$u(t) = e^{-cLt}u(0),$$

onde $u(0)$ é a condição inicial e e^{-cLt} é a exponencial matricial. Como L é simétrica, admite uma base ortonormal de autovetores. Se $Lv_i = \lambda_i v_i$ e $u(0) = \sum_{i=1}^{N} \alpha_i v_i$, com $\alpha_i = v_i^T u(0)$, então

$$u(t) = \sum_{i=1}^{N} \alpha_i e^{-c\lambda_i t} v_i.$$

Cada modo espectral evolui com fator $e^{-c\lambda_i t}$. O modo associado ao autovalor nulo permanece inalterado, enquanto os modos associados a autovalores positivos decaem. Para um grafo conexo, $\lambda_1 = 0$ é simples e o segundo menor autovalor λ_2 determina a taxa assintótica mais lenta de dissipação das diferenças em relação ao equilíbrio.

7.4 Método de Euler explícito

Considere instantes discretos $t_k = k\Delta t$, com passo temporal $\Delta t > 0$. Aplicando Euler explícito ao sistema $\dot{u}(t) = -cLu(t)$,

$$\frac{u^{k+1} - u^k}{\Delta t} \approx -cLu^k.$$

Assim,

$$u^{k+1} = u^k - c\Delta t L u^k = (I - c\Delta t L)u^k.$$

Componente a componente,

$$\mathrm{u}_i^{k+1} = u_i^k + c\Delta t \sum_{v_j \sim v_i} (u_j^k - u_i^k).$$

Essa é a regra utilizada pela implementação. Todos os novos valores são calculados usando exclusivamente o estado anterior, armazenados separadamente e atribuídos simultaneamente ao final do passo. Desse modo, o resultado não depende da ordem em que os vértices são percorridos.

O histórico pode ser representado por $u^0, u^1, \ldots, u^K$, em que u^0 é a condição inicial e u^k corresponde ao estado após k passos. O instante físico é $t_k = k\Delta t$. Esse passo temporal não deve ser confundido com o intervalo, em milissegundos, usado para atualizar a animação da interface: este último controla somente a frequência de exibição dos quadros.

7.5 Estabilidade numérica

Para um autovetor associado a λ_i, um passo de Euler multiplica sua amplitude pelo fator

$$\mu_i = 1 - c\Delta t \lambda_i.$$

A condição para que nenhum modo seja amplificado em módulo é $|\mu_i| \leq 1$. Como os autovalores da Laplaciana são não negativos, obtém-se

$$0 \leq c\Delta t \lambda_i \leq 2.$$

Basta, portanto, verificar o maior autovalor:

$$\boxed{0 \leq c\Delta t \lambda_{\max} \leq 2.}$$

Para amortecimento estrito de todos os modos não constantes, utiliza-se desigualdade estrita no limite superior; na igualdade, um modo pode apresentar fator -1 e não decair.

Na rede do experimento, a análise espectral forneceu $\lambda_{\max} \approx 11{,}72$. Nos vídeos, manteve-se $\Delta t = 1{,}0$ e foram utilizados dois valores de c:

c	Δt	$c\Delta t\lambda_{\max}$
0,01	1,0	0,1172
0,08	1,0	0,9376

Ambas as configurações satisfazem a condição de estabilidade, pois os produtos são menores que 2. O aumento de c acelera a redistribuição no modelo contínuo, mas também aumenta $c\Delta t\lambda_{\max}$ e reduz a margem de estabilidade do método explícito.

É necessário distinguir as condições iniciais documentadas. A inicialização padrão do script atribui valor zero a todos os vértices, produzindo um estado uniforme que não evolui. Nos vídeos, foi aplicada uma condição não uniforme: o vértice de rótulo 15 recebeu 100 °C e os demais começaram em 0 °C. Essa diferença explica por que a difusão é observável nos materiais experimentais.

7.6 Equação do calor e stencil de cinco pontos

A difusão em grafos relaciona-se à equação do calor em uma região bidimensional:

$$\frac{\partial u}{\partial t} = \kappa \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right),$$

onde $\kappa > 0$ é a difusividade térmica. Em uma malha quadrada de espaçamento h, as derivadas espaciais podem ser aproximadas por diferenças centrais:

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2}, \qquad \frac{\partial^2 u}{\partial y^2} \approx \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h^2}.$$

Somando as aproximações, obtém-se o stencil de cinco pontos:

$$\frac{du_{i,j}}{dt} \approx \frac{\kappa}{h^2} \left(u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j} \right).$$

O stencil envolve o ponto central e seus quatro vizinhos ortogonais: acima, abaixo, à esquerda e à direita. Isso corresponde a representar a malha por um grafo em que cada vértice interior se conecta aos quatro pontos adjacentes.

Para vértices interiores de grau quatro, a expressão entre parênteses é o negativo da componente correspondente da Laplaciana do grafo da malha. Assim, a discretização espacial pode ser escrita como

$$\frac{du}{dt} = -\frac{\kappa}{h^2} Lu.$$

Ela tem a mesma estrutura do modelo de difusão em grafos, com a identificação $c = \kappa/h^2$ para essa malha regular e essa convenção de pesos.

Essa correspondência mostra que a difusão em grafos estende a ideia de difusão para redes cuja conectividade não precisa seguir uma geometria regular. Na malha, as arestas representam vizinhanças espaciais; em um grafo geral, representam relações

de vizinhança definidas pela própria rede. No experimento deste trabalho, as arestas são não ponderadas e cada vizinho contribui com o mesmo coeficiente de troca. Portanto, a rede aleatória estudada não deve ser interpretada como uma malha geométrica: o stencil de cinco pontos é apresentado como conexão conceitual com a discretização espacial clássica da equação do calor.

8 Experimento em redes com comunidades

O experimento investiga como a estrutura de uma rede com duas comunidades influencia a difusão de calor entre seus vértices. Cada vértice representa uma posição na qual se armazena uma temperatura, e cada aresta representa uma possibilidade de troca direta de calor. A dinâmica segue o modelo Laplaciano apresentado na fundamentação teórica, discretizado pelo método de Euler explícito.

É importante distinguir a configuração reproduzível definida pelo script de teste dos valores que podem ser alterados durante a execução interativa. O script fornecido constrói uma rede com parâmetros fixos e abre o laboratório gráfico; durante a interação, o usuário pode selecionar vértices, modificar temperaturas e alterar parâmetros da simulação.

8.1 Construção da rede

A rede é gerada pelo modelo aleatório de duas comunidades implementado na função `criar_grafo_comunidades`. Sejam A e B os conjuntos de vértices das comunidades. O procedimento recebe os tamanhos n e m, a probabilidade p de conexão entre vértices pertencentes à mesma comunidade e a probabilidade q de conexão entre vértices de comunidades distintas.

No script de teste, são utilizados $n = m = 10$, de modo que

$$|A| = |B| = 10, \qquad |V| = 20.$$

Os vértices da primeira comunidade são identificados pelos inteiros de 0 a 9, enquanto os da segunda comunidade são identificados de 10 a 19. Para cada par não ordenado de vértices, a aresta é incluída por um sorteio independente: utiliza-se $p = 0{,}80$ para pares internos a uma mesma comunidade e $q = 0{,}05$ para pares entre comunidades. O grafo resultante é simples e não direcionado, sendo armazenado por listas de adjacência.

A semente pseudoaleatória definida no script é `seed=42`. Assim, mantidos o algoritmo e os parâmetros, a geração da rede pode ser reproduzida. A semente controla a realização aleatória, mas não transforma as probabilidades p e q em quantidades determinísticas: a quantidade efetiva de arestas depende do sorteio.

Na realização obtida com os parâmetros do script, a rede possui 76 arestas: 36 internas à comunidade A, 37 internas à comunidade B e 3 entre comunidades. A rede gerada é conexa. Esses valores são propriedades calculadas para a realização associada à semente 42, e não valores esperados para todas as redes do mesmo modelo. A quantidade esperada de arestas, antes da realização dos sorteios, é

$$\mathbb{E}[|E|] = p\left[\binom{n}{2} + \binom{m}{2}\right] + qnm.$$

Para $n = m = 10$, $p = 0{,}80$ e $q = 0{,}05$, essa expressão fornece 58,5 arestas esperadas. A diferença entre a esperança e as 76 arestas observadas é compatível com a natureza aleatória da construção.

8.2 Cenários experimentais

A comparação de interesse consiste em manter fixa a rede e a condição inicial e variar a constante de difusão c. Um valor menor de c representa uma taxa de troca mais lenta por unidade de tempo, enquanto um valor maior tende a acelerar a redução das diferenças de temperatura, desde que o passo numérico permaneça estável.

O código disponibilizado não contém uma rotina automatizada que execute uma bateria de cenários, salve seus resultados e produza uma tabela comparativa. Assim, a comparação entre diferentes valores de c é realizada por meio do controle deslizante do laboratório interativo. O intervalo configurado para esse controle é de 0 a 0,10, com incremento de 0,001.

Nos dois vídeos, a rede utilizada é mantida fixa e a condição inicial consiste em atribuir temperatura de 100 °C ao vértice 15, enquanto os demais vértices começam a 0 °C. O vértice 15 pertence à comunidade B. Dessa forma, inicialmente, a temperatura média da comunidade A é 0 °C, enquanto a temperatura média da comunidade B é 10 °C. A temperatura média global da rede é 5 °C.

A diferença entre as gravações está no valor da constante de difusão. No primeiro vídeo, utiliza-se $c = 0{,}01$, enquanto, no segundo, utiliza-se $c = 0{,}08$. No segundo cenário, o passo temporal é $\Delta t = 1{,}00$, com execução de até 1000 iterações.

O script de teste abre o laboratório com $c = 0{,}01$, valor também utilizado no primeiro vídeo. Já a configuração $c = 0{,}08$ é selecionada por meio do controle deslizante e utilizada na segunda gravação. Portanto, os valores empregados nos dois cenários estão documentados nos respectivos materiais experimentais.

Para que a comparação seja controlada, devem ser mantidas a mesma realização do grafo, a mesma condição inicial e o mesmo passo temporal, alterando-se somente a constante de difusão c. Quando forem utilizadas redes geradas em mais de uma execução, a semente aleatória de cada realização também deve ser registrada.

A comparação entre os vídeos permite observar visualmente a influência da constante de difusão na velocidade de redistribuição da temperatura. Com $c = 0{,}08$, espera-se uma redistribuição mais rápida do que com $c = 0{,}01$, pois a taxa de difusão é maior.

8.3 Condição inicial e parâmetros

No script de teste, o dicionário de temperaturas é inicializado com temperatura nula em todos os vértices:

$$u_i(0) = 0, \qquad i = 1, \ldots, 20.$$

Essa inicialização uniforme corresponde a um estado de equilíbrio térmico, uma vez que todos os vértices apresentam a mesma temperatura. Portanto, não ocorre evolução térmica enquanto nenhuma temperatura for modificada. O próprio comentário do script indica que a intenção é testar temperaturas diferentes; na interface do laboratório, essa alteração é realizada por meio da seleção de um vértice e da aplicação de um valor informado pelo usuário.

Nos vídeos experimentais, observa-se o vértice 15 com temperatura de 100 °C, enquanto os demais vértices exibidos apresentam inicialmente temperatura de 0 °C. Como os vértices 10 a 19 pertencem ao grupo B, essa configuração corresponde à aplicação de uma fonte térmica localizada nessa comunidade. Considerando exatamente essa condição inicial, a média das temperaturas da comunidade A é 0 °C, enquanto a média da comunidade B é 10 °C, pois apenas um de seus dez vértices apresenta temperatura diferente de zero. Consequentemente, a média global da rede é 5 °C. Essa configuração corresponde ao estado visualizado nos vídeos e deve ser distinguida da inicialização literal do script, que atribui temperatura nula a todos os vértices.

Na chamada de teste do script, são utilizados os parâmetros $c = 0{,}01$, $\Delta t = 1{,}0$, 200 passos de simulação e intervalo de animação de 100 milissegundos. O laboratório interativo permite modificar o número máximo de passos, a constante de difusão c, o passo temporal Δt e a velocidade de reprodução da animação. Além disso, a caixa de texto permite definir a temperatura aplicada ao vértice selecionado, possibilitando alterar a condição inicial do sistema.

Para o experimento apresentado no Vídeo 2, foram utilizados 1000 passos de simulação, $\Delta t = 1{,}00$ e $c = 0{,}08$. Nesse caso, foi aplicada uma temperatura de 100 °C ao vértice selecionado, identificado como vértice 15, enquanto os demais vértices permaneceram inicialmente a 0 °C. Assim, o experimento parte de uma condição inicial não uniforme, com uma fonte térmica localizada na comunidade B, permitindo acompanhar a propagação do calor pela rede. Em comparação com o Vídeo 1, realizado com $c = 0{,}01$, o valor $c = 0{,}08$ representa uma intensidade de difusão maior e, consequentemente, uma tendência de equalização térmica mais rápida, desde que respeitada a condição de estabilidade do método numérico.

8.4 Simulação e repetição

A cada passo, a implementação calcula, para todos os vértices, a soma das diferenças entre a temperatura do vértice e as temperaturas de seus vizinhos. Todos os novos valores são calculados a partir do mesmo estado anterior e somente depois são atribuídos simultaneamente. Essa atualização evita que a ordem de percurso dos vértices altere o resultado de um mesmo passo.

O histórico completo de estados é armazenado pela função `simular_troca_calor`. Já o laboratório interativo mantém o estado corrente e registra, ao longo da animação, as médias das comunidades. O botão de reinicialização retorna à distribuição salva como referência; após a aplicação manual de uma temperatura, essa distribuição de reinicialização é atualizada.

A semente 42 permite reproduzir a topologia gerada pelo script de teste. Entretanto, os materiais fornecidos não apresentam uma rotina de repetição em lote, agregação estatística entre várias sementes ou exportação automática dos históricos para arquivos. Portanto, não se afirma que tenham sido realizadas repetições independentes ou estimativas de incerteza. Uma avaliação estatística mais ampla exigiria executar explicitamente várias sementes e armazenar os resultados de cada execução.

9 Métricas para análise da dinâmica

As métricas a seguir formalizam medidas adequadas para acompanhar a homogeneização térmica. Deve-se observar que nem todas estão implementadas no código fornecido: o laboratório registra as médias de temperatura das comunidades e apresenta suas curvas; as demais expressões são definições para análise e não devem ser confundidas com resultados já produzidos automaticamente pela interface.

9.1 Temperatura média

Para uma rede com $N = |V|$ vértices, a temperatura média global no instante t é

$$\overline{u}(t) = \frac{1}{N} \sum_{i=1}^{N} u_i(t).$$

Em um grafo não direcionado, sem fontes ou sumidouros externos e com a dinâmica baseada no Laplaciano combinatório, a soma das temperaturas é conservada no modelo contínuo, pois $\mathbf{1}^{\mathsf{T}} L = 0$. A atualização explícita também conserva essa soma em aritmética exata, uma vez que cada aresta contribui com termos opostos para seus dois extremos. A implementação do laboratório não calcula nem plota diretamente a média global; essa métrica pode ser obtida a partir do vetor de temperaturas em cada passo.

9.2 Temperatura média das comunidades

Se A e B são as comunidades e seus tamanhos são não nulos, definem-se

$$\overline{u}_A(t) = \frac{1}{|A|} \sum_{i \in A} u_i(t), \qquad \overline{u}_B(t) = \frac{1}{|B|} \sum_{i \in B} u_i(t).$$

A função `temperatura_media_grupo` implementa o cálculo da média de um conjunto de vértices. No laboratório, as médias dos grupos são registradas a cada passo e representadas em um gráfico em função do tempo simulado. A legenda identifica as curvas como Grupo A e Grupo B.

9.3 Diferença entre comunidades

A diferença entre as médias das comunidades pode ser acompanhada por

$$\Delta_{AB}(t) = \left| \overline{u}_A(t) - \overline{u}_B(t) \right|.$$

O decaimento dessa quantidade indica aproximação entre as médias dos dois grupos. Entretanto, igualdade entre as médias comunitárias não implica, por si só, que todos os vértices tenham a mesma temperatura: podem persistir diferenças internas em cada comunidade. A interface apresenta as duas médias, mas não calcula nem desenha explicitamente a curva $\Delta_{AB}(t)$.

9.4 Variância do estado da rede

A dispersão das temperaturas em relação à média global pode ser medida por

$$\sigma^2(t) = \frac{1}{N} \sum_{i=1}^{N} \left(u_i(t) - \overline{u}(t)\right)^2 .$$

Uma variância nula equivale a temperaturas iguais em todos os vértices. Para uma rede conexa, esse estado é o equilíbrio homogêneo do modelo de difusão considerado. A variância não é calculada nem exibida pelo código fornecido; sua inclusão nesta seção define uma métrica recomendada para uma análise quantitativa posterior.

9.5 Tempo de convergência

Um critério operacional possível é definir a convergência pelo alcance de uma dispersão máxima inferior a uma tolerância ε:

$$\max_i u_i(t) - \min_i u_i(t) < \varepsilon.$$

No botão *Até Equilíbrio*, a implementação utiliza a tolerância 0,01 e interrompe a busca quando a amplitude entre a maior e a menor temperatura fica abaixo desse valor. Há também um limite de 5000 passos nessa rotina. Se o limite for alcançado antes da condição, o código não verifica explicitamente e comunica uma falha de convergência; a mensagem exibida continua sendo "Equilíbrio atingido". Assim, para uma avaliação rigorosa, é necessário verificar também se o critério foi efetivamente satisfeito ao término.

O tempo simulado correspondente a k passos é $t_k = k\Delta t$. A interface mostra o número de passos e o tempo simulado, mas não armazena um indicador independente de convergência nem compara automaticamente tempos de convergência entre cenários. Além disso, o critério de amplitude global é mais forte do que a simples igualdade entre médias de comunidades.

9.6 Características espectrais

A matriz Laplaciana combinatória é $L = D - A$. Para o grafo não direcionado considerado, L é simétrica e semidefinida positiva. Seus autovalores reais podem ser ordenados como

$$0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_N.$$

A multiplicidade do autovalor zero corresponde ao número de componentes conexas. Quando o grafo é conexo, $\lambda_2 > 0$; esse segundo menor autovalor, denominado conectividade algébrica, está relacionado à escala temporal de relaxação do modo mais lento não constante. O maior autovalor também importa para a estabilidade do método explícito.

A implementação fornecida constrói a matriz Laplaciana, mas não contém uma rotina de cálculo de autovalores ou autovetores. Como verificação numérica externa aplicada à realização reproduzida com a semente 42, a diagonalização da matriz 20×20 fornece, aproximadamente,

$$\lambda_2 \approx 0{,}4711, \qquad \lambda_{20} \approx 11{,}7200.$$

Esses números são resultados de pós-processamento da rede gerada, não saídas da interface original. A existência de um único autovalor numericamente nulo, juntamente com a conectividade verificada por busca em largura, é consistente com a existência de uma única componente conexa.

Para a dinâmica contínua, os modos associados aos autovalores decaem como $e^{-c\lambda_i t}$. Portanto, $c\lambda_2$ caracteriza a taxa de decaimento do modo não constante mais lento. Para a discretização explícita, a atualização é estável para todos os modos se

$$0 < c\Delta t\, \lambda_N < 2.$$

Essa condição deve ser verificada para cada combinação de c, Δt e rede. A interface permite alterar c e Δt, mas não calcula automaticamente λ_N nem impede a escolha de parâmetros que violem a condição de estabilidade.

10 Visualização e animação

A visualização principal organiza os vértices em uma circunferência, desenha as arestas em cinza e representa as temperaturas por uma escala de cores. Os rótulos exibem o identificador do vértice e sua temperatura. A barra de cores fornece a referência visual para interpretar os valores térmicos. A disposição circular é uma escolha de apresentação: não representa distâncias físicas entre vértices.

O laboratório interativo utiliza a biblioteca Matplotlib e disponibiliza controles para iniciar ou pausar a animação, reiniciar o estado, alterar o número de passos, modificar c, modificar Δt e ajustar a velocidade de reprodução. Também é possível selecionar um vértice diretamente no desenho, digitar uma nova temperatura e aplicar a alteração. O gráfico lateral apresenta a temperatura média de cada comunidade ao longo do tempo simulado.

A animação permite observar qualitativamente a transferência de calor pela rede e comparar a rapidez de evolução ao variar c. Para uma comparação válida, devem ser mantidos constantes a rede, a condição inicial e o passo temporal. A escala de cores e os limites do gráfico também devem ser considerados ao interpretar visualmente as diferenças entre gravações.

Vídeos do experimento

Para avaliar a influência da constante de difusão na evolução do sistema, foram realizados dois experimentos, apresentados nos vídeos a seguir. No primeiro experimento, correspondente ao Vídeo 1, foi utilizado o valor $c = 0{,}01$, enquanto no segundo experimento, correspondente ao Vídeo 2, foi utilizado $c = 0{,}08$. Portanto, o primeiro cenário representa a condição de menor difusão, e o segundo, a condição de maior difusão. A comparação entre as animações permite observar que, com $c = 0{,}01$, a transferência de calor ocorre de maneira mais gradual, fazendo com que as diferenças de temperatura persistam por mais tempo. Já com $c = 0{,}08$, a difusão ocorre mais rapidamente, acelerando a aproximação das temperaturas dos vértices. Esse comportamento é coerente com o modelo de difusão adotado, no qual a constante c controla a intensidade da transferência de calor entre os vértices conectados. A comparação é qualitativa e deve ser interpretada respeitando a condição de estabilidade do método numérico.

Vídeo 1 — Difusão com menor constante de difusão.

`https://youtu.be/Lr2yGAdHH3k?si=zciW1dCcXIVwUiNA`

Vídeo 2 — Difusão com maior constante de difusão.

`https://youtu.be/ilsog85FO7M?si=zcxFg3WGsxk2U10e`

11 Resultados

A comparação dos dois vídeos experimentais evidencia a influência da constante de difusão (c) na velocidade de equalização térmica da rede. No primeiro cenário, com menor valor de (c), a transferência de calor entre os vértices ocorre de maneira mais gradual, fazendo com que as diferenças de temperatura persistam por mais tempo. No segundo cenário, com maior valor de (c), a difusão ocorre mais rapidamente, levando a uma aproximação mais acelerada entre as temperaturas dos vértices.

Esse comportamento está de acordo com a dinâmica descrita pela equação de difusão, uma vez que a constante (c) controla a intensidade da transferência de calor. Assim, quanto maior seu valor, maior tende a ser a taxa de redução das diferenças de temperatura, respeitada a condição de estabilidade do método numérico. A comparação visual das animações confirma qualitativamente essa relação entre a constante de difusão e a velocidade de equalização térmica.

12 Discussão

A construção probabilística produz comunidades densamente conectadas internamente e fracamente conectadas entre si em termos de probabilidades: $p = 0{,}80$ é muito maior que $q = 0{,}05$. Na realização específica, a existência de três arestas entre comunidades mantém o grafo conexo, ao mesmo tempo que a separação estrutural entre os grupos pode criar modos de difusão associados à transferência entre comunidades. A conectividade algébrica positiva confirma que há um caminho de comunicação entre todos os vértices; seu valor relativamente menor que os demais autovalores não nulos é compatível com um modo de relaxação mais lento. Essa interpretação está de acordo com a estrutura espectral da matriz Laplaciana obtida para a rede analisada.

A matriz Laplaciana reúne a informação da topologia e determina a dinâmica por meio de

$$\dot{u}(t) = -cLu(t).$$

Mantendo o grafo e a condição inicial, o aumento de c acelera o decaimento dos modos não constantes na solução contínua. No método de Euler explícito, entretanto, a estabilidade também depende do passo temporal. Para a discretização

$$u^{k+1} = (I - c\Delta t L)u^k,$$

a condição de estabilidade dos modos associados aos autovalores da Laplaciana é

$$0 \leq c\Delta t\lambda_i \leq 2.$$

Na rede analisada, o maior autovalor é aproximadamente $\lambda_N = 11{,}72$. Com $\Delta t = 1{,}0$, os parâmetros utilizados nos vídeos produzem $c\Delta t\lambda_N \approx 0{,}1172$, para $c = 0{,}01$,

e $c\Delta t\lambda_N \approx 0{,}9376$, para $c = 0{,}08$. Ambos os valores satisfazem a condição de estabilidade do método explícito para essa matriz. Assim, a diferença observada entre os vídeos pode ser interpretada como efeito da alteração do parâmetro de difusão, sem que os parâmetros informados indiquem instabilidade numérica nessa configuração.

A implementação apresenta limitações importantes para a interpretação dos resultados. A inicialização padrão do script de teste atribui temperatura nula a todos os vértices e, sem uma alteração manual, não produz difusão observável, pois o estado inicial já é uniforme. Nos vídeos, essa condição foi modificada: o vértice 15 foi inicializado a 100 °C, enquanto os demais permaneceram a 0 °C, criando um gradiente de temperatura que permite observar a evolução do sistema. A interface registra as médias das comunidades, mas não implementa diretamente todas as métricas que poderiam aprofundar a análise, como séries temporais completas da variância e da diferença entre as médias comunitárias. Além disso, o botão "Até Equilíbrio" utiliza a amplitude máxima-mínima como critério e impõe um limite de iterações, mas não distingue explicitamente a satisfação do critério do esgotamento desse limite. A interface também não calcula o espectro da Laplaciana nem verifica automaticamente a estabilidade antes de executar a simulação. Por fim, o modelo trata as conexões como trocas homogêneas, sem pesos, capacidades térmicas distintas, fontes permanentes ou perdas para o ambiente.

Os dois vídeos documentam configurações distintas do experimento: no primeiro, utiliza-se $c = 0{,}01$; no segundo, $c = 0{,}08$, mantendo-se $\Delta t = 1{,}0$. No segundo cenário, a simulação foi executada por 1000 passos. A comparação visual é compatível com a previsão do modelo: o valor maior de c promove uma redistribuição de temperatura mais rápida ao longo da rede. Essa comparação, contudo, é qualitativa. Como não foram registrados e analisados históricos numéricos completos para os dois cenários, não se atribuem aos vídeos tempos medidos de convergência nem estimativas quantitativas da diferença entre as taxas de relaxação. Portanto, os resultados permitem relacionar o comportamento visual observado à dinâmica Laplaciana e aos parâmetros documentados, mas não sustentam uma comparação quantitativa de desempenho entre as duas execuções.

13 Conclusões finais e trabalhos futuros

Este trabalho teve como objetivo estudar conceitos fundamentais da teoria dos grafos e investigar sua aplicação na modelagem de sistemas dinâmicos em redes. Para isso, foram desenvolvidos e analisados algoritmos de manipulação de grafos, métodos de busca, construção de árvores geradoras mínimas e ferramentas de análise matricial e espectral. Esses conceitos foram posteriormente aplicados ao estudo da difusão de calor em uma rede de vértices interconectados.

A implementação dos algoritmos permitiu explorar diferentes propriedades estruturais dos grafos, incluindo conectividade, caminhos, componentes e árvores geradoras mínimas. A utilização das matrizes de adjacência, grau e Laplaciana possibilitou representar matematicamente a estrutura da rede e estabelecer a relação entre a topologia do grafo e o comportamento de sistemas dinâmicos definidos sobre seus vértices.

Na etapa experimental, foi construída uma rede aleatória com duas comunidades, permitindo observar como a organização das conexões influencia a dinâmica de difusão.

A análise espectral da matriz Laplaciana apresentou um segundo menor autovalor positivo, indicando que a rede gerada era conexa. Esse resultado também reforçou a importância das propriedades espectrais para a compreensão do comportamento global de sistemas dinâmicos em grafos.

A simulação da difusão de calor, realizada por meio do método de Euler explícito, permitiu acompanhar a evolução das temperaturas dos vértices ao longo do tempo. A comparação entre os dois vídeos experimentais, produzidos com diferentes valores da constante de difusão, evidenciou qualitativamente a influência desse parâmetro na velocidade de equalização térmica. Observou-se que, para um valor menor de difusão, as diferenças de temperatura persistem por mais tempo, enquanto um valor maior promove uma aproximação mais rápida entre as temperaturas dos vértices. Esse comportamento está de acordo com o modelo matemático adotado e demonstra a relação entre os parâmetros do sistema e sua evolução temporal.

Apesar dos resultados obtidos, o estudo apresenta algumas limitações. A análise experimental foi realizada a partir de uma configuração específica de rede, de modo que diferentes topologias e sementes aleatórias poderiam produzir comportamentos distintos. Além disso, a comparação entre os cenários de difusão foi principalmente qualitativa, baseada na observação das animações, sem uma análise sistemática dos tempos necessários para atingir o equilíbrio térmico. Dessa forma, os resultados permitem identificar tendências, mas não estabelecem uma comparação quantitativa abrangente entre diferentes configurações.

Como possibilidades de trabalhos futuros, destacam-se a realização de experimentos com diferentes topologias, tamanhos de rede e parâmetros de geração, bem como a comparação de múltiplas realizações de grafos aleatórios. Também seria interessante incorporar métricas quantitativas à análise da difusão, como a variância das temperaturas, a diferença entre as temperaturas médias das comunidades e o tempo necessário para atingir uma determinada tolerância de equilíbrio. Outra extensão possível consiste em investigar a influência de diferentes valores da constante de difusão e do passo temporal sobre a estabilidade e a velocidade de convergência do método numérico.

Referências

[1] Nair Maria Maia de Abreu, Renata Raposo Del-Vecchio, Cybele Tavares Maia Vinagre, and Dragan Stevanović. *Introdução à Teoria Espectral de Grafos com Aplicações*, volume 27 of *Notas em Matemática Aplicada*. Sociedade Brasileira de Matemática Aplicada e Computacional, São Carlos, SP, 2 edition, 2012.

[2] Fan R. K. Chung. *Spectral Graph Theory*, volume 92 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, Providence, RI, 1997.

[3] Reinhard Diestel. *Graph Theory*, volume 173 of *Graduate Texts in Mathematics*. Springer, Berlin, Heidelberg, 6 edition, 2025.

A Códigos desenvolvidos

Este apêndice reúne recortes representativos dos códigos utilizados no desenvolvimento do trabalho e na realização dos experimentos. Os trechos estão organizados

por funcionalidade e complementados por uma documentação técnica do laboratório interativo. Quando um procedimento é apresentado apenas como análise complementar ou sugestão de aperfeiçoamento, essa condição é explicitada para não confundi-lo com uma funcionalidade já existente no programa original.

A.1 Classe Grafo

```
class Grafo:

    def __init__(self):
        self.adjacencia = {}

    def adicionar_vertice(self, vertice):
        if vertice not in self.adjacencia:
            self.adjacencia[vertice] = []

    def remover_vertice(self, vertice):

        if vertice not in self.adjacencia:
            return

        for vizinho in self.adjacencia[vertice]:
            self.adjacencia[vizinho].remove(vertice)

        del self.adjacencia[vertice]

    def existe_vertice(self, vertice):
        return vertice in self.adjacencia

    def adicionar_aresta(self, u, v):

        if u == v:
            raise ValueError(
                "Um grafo simples não permite laços."
            )

        self.adicionar_vertice(u)
        self.adicionar_vertice(v)

        if v not in self.adjacencia[u]:
            self.adjacencia[u].append(v)

        if u not in self.adjacencia[v]:
            self.adjacencia[v].append(u)

    def remover_aresta(self, u, v):

        if u in self.adjacencia and v in self.adjacencia[u]:
            self.adjacencia[u].remove(v)

        if v in self.adjacencia and u in self.adjacencia[v]:
```

```
45              self.adjacencia[v].remove(u)
46
47      def existe_aresta(self, u, v):
48          return (
49              u in self.adjacencia
50              and v in self.adjacencia[u]
51          )
52
53      def vizinhos(self, vertice):
54
55          if vertice not in self.adjacencia:
56              return []
57
58          return self.adjacencia[vertice].copy()
59
60      def grau(self, vertice):
61
62          if vertice not in self.adjacencia:
63              return 0
64
65          return len(self.adjacencia[vertice])
66
67      def quantidade_vertices(self):
68          return len(self.adjacencia)
69
70      def quantidade_arestas(self):
71
72          total = sum(
73              len(vizinhos)
74              for vizinhos in self.adjacencia.values()
75          )
76
77          return total // 2
```

A.2 Algoritmos BFS e DFS

A implementação utilizada é:

```
1  from collections import deque
2
3  def bfs(self, inicio):
4
5      if inicio not in self.adjacencia:
6          return [], {}, {}
7
8      fila = deque([inicio])
9
10     visitados = {inicio}
11
12     ordem = []
13
```

```
    distancia = {
        inicio: 0
    }

    predecessor = {
        inicio: None
    }

    while fila:

        atual = fila.popleft()

        ordem.append(atual)

        for vizinho in self.adjacencia[atual]:

            if vizinho not in visitados:

                visitados.add(vizinho)

                fila.append(vizinho)

                distancia[vizinho] = (
                    distancia[atual] + 1
                )

                predecessor[vizinho] = atual

    return ordem, distancia, predecessor
```

A.3 Trechos de implementação apresentados no desenvolvimento

Os trechos a seguir correspondem às implementações utilizadas para os algoritmos e procedimentos descritos no corpo do trabalho. Eles são apresentados no apêndice para manter a fundamentação e a discussão centradas nos conceitos matemáticos e nos algoritmos, deixando os detalhes de programação reunidos em um único local.

A.3.1 Busca em largura – reconstrução de caminho

```
def caminho_bfs(self, inicio, destino):

    if (
        inicio not in self.adjacencia
        or destino not in self.adjacencia
    ):
        return []

    ordem, distancia, predecessor = self.bfs(inicio)

```

```
    if destino not in distancia:
        return []

    caminho = []

    atual = destino

    while atual is not None:

        caminho.append(atual)

        atual = predecessor[atual]

    caminho.reverse()

    return caminho
```

A.3.2 Busca em profundidade

```
def dfs(self, inicio):

    if inicio not in self.adjacencia:
        return [], {}

    visitados = set()

    ordem = []

    predecessor = {
        inicio: None
    }

    def visitar(atual):

        visitados.add(atual)

        ordem.append(atual)

        for vizinho in self.adjacencia[atual]:

            if vizinho not in visitados:

                predecessor[vizinho] = atual

                visitar(vizinho)

    visitar(inicio)

    return ordem, predecessor
```

A.3.3 Conectividade

```
def eh_conexo(self):

    if not self.adjacencia:
        return True

    inicio = next(iter(self.adjacencia))

    ordem, _, _ = self.bfs(inicio)

    return len(ordem) == self.quantidade_vertices()
```

A.3.4 Componentes conexas

```
def componentes_conexas(self):

    visitados = set()

    componentes = []

    for inicio in self.adjacencia:

        if inicio in visitados:
            continue

        ordem, _, _ = self.bfs(inicio)

        componentes.append(ordem)

        visitados.update(ordem)

    return componentes
```

A.3.5 Geração de ciclos

```
def criar_circuito(n):

    validar_n(n)

    if n < 3:
        raise ValueError(
            "Um ciclo simples deve possuir pelo menos 3 vé
               rtices."
        )

    grafo = Grafo()

    for i in range(n):
```

```
        grafo.adicionar_vertice(i)

    for i in range(n):

        proximo = (i + 1) % n

        grafo.adicionar_aresta(
            i,
            proximo
        )

    return grafo
```

A.3.6 Geração de grafos completos

```
def criar_grafo_completo(n):

    validar_n(n)

    grafo = Grafo()

    for i in range(n):
        grafo.adicionar_vertice(i)

    for i in range(n):

        for j in range(i + 1, n):

            grafo.adicionar_aresta(i, j)

    return grafo
```

A.3.7 Geração de grafos bipartidos

```
def criar_grafo_bipartido(n, m):

    validar_n(n)
    validar_n(m)

    grafo = Grafo()

    parte_a = list(range(n))
    parte_b = list(range(n, n + m))

    for vertice in parte_a:
        grafo.adicionar_vertice(vertice)

    for vertice in parte_b:
        grafo.adicionar_vertice(vertice)
```

```
16
17      for u in parte_a:
18
19          for v in parte_b:
20
21              grafo.adicionar_aresta(u, v)
22
23      return grafo
```

A.3.8 Geração de grafos aleatórios

```
1  def criar_grafo_aleatorio(n, p, seed=None):
2
3      validar_n(n)
4
5      if not 0 <= p <= 1:
6          raise ValueError(
7              "A probabilidade p deve estar entre 0 e 1."
8          )
9
10     gerador = random.Random(seed)
11
12     grafo = Grafo()
13
14     for i in range(n):
15         grafo.adicionar_vertice(i)
16
17     for i in range(n):
18
19         for j in range(i + 1, n):
20
21             if gerador.random() < p:
22                 grafo.adicionar_aresta(i, j)
23
24     return grafo
```

A.3.9 Exemplo de geração de grafo aleatório

```
1  grafo = criar_grafo_aleatorio(
2      n=8,
3      p=0.3,
4      seed=42
5  )
```

A.3.10 Geração paramétrica

```
1  def criar_grafo_parametrico(n, tipo="completo"):
2
```

```
    tipo = tipo.lower()

    if tipo == "ciclo":
        return criar_circuito(n)

    elif tipo == "completo":
        return criar_grafo_completo(n)

    else:
        raise ValueError(
            "Tipo de grafo desconhecido. "
            "Use 'ciclo' ou 'completo'."
        )
```

A.3.11 Visualização dos grafos

```
vertices = list(grafo.adjacencia.keys())

n = len(vertices)

posicoes = {}

for i, vertice in enumerate(vertices):

    angulo = 2 * math.pi * i / n

    x = math.cos(angulo)
    y = math.sin(angulo)

    posicoes[vertice] = (x, y)
```

```
arestas_desenhadas = set()

for u in grafo.adjacencia:

    for v in grafo.adjacencia[u]:

        aresta = frozenset((u, v))

        if aresta not in arestas_desenhadas:

            x1, y1 = posicoes[u]
            x2, y2 = posicoes[v]

            plt.plot(
                [x1, x2],
                [y1, y2],
                color="black",
                linewidth=1.5
            )
```

```
20
21                arestas_desenhadas.add(aresta)
```

A.3.12 Salvamento das figuras

```
pasta_codigo = os.path.dirname(
    os.path.abspath(__file__)
)

caminho_imagem = os.path.join(
    pasta_codigo,
    nome_arquivo
)
```

```
plt.savefig(
    caminho_imagem,
    dpi=300,
    bbox_inches="tight"
)
```

A.4 Organização e escopo dos códigos apresentados

Os trechos desta primeira parte documentam a estrutura de dados do grafo, as operações básicas sobre vértices e arestas, as buscas em largura e profundidade, a reconstrução de caminhos, a verificação de conectividade, a identificação de componentes conexas e alguns procedimentos de geração e visualização de grafos.

A construção específica da rede com duas comunidades, a integração temporal por Euler explícito, os controles da interface, o registro das médias comunitárias e o pós-processamento espectral são documentados na Seção B.

É importante delimitar o que está efetivamente documentado pelos arquivos fornecidos: eles não incluem, nos trechos analisados, uma implementação do algoritmo de Kruskal com a estrutura Union-Find, nem uma rotina própria para calcular autovetores. O cálculo dos autovalores apresentado adiante é um pós-processamento independente da matriz Laplaciana. Da mesma forma, a interface original registra as médias dos grupos, mas não calcula automaticamente a variância global, a diferença absoluta entre as médias ou o tempo de convergência como métricas armazenadas. Esses limites são explicitados para que o apêndice não atribua ao programa funcionalidades que não estão presentes no código analisado.

B Documentação técnica da implementação do experimento

Este apêndice documenta os procedimentos efetivamente presentes nos arquivos Python fornecidos. A implementação principal está no módulo `irere_temperaturaanim.py`; o arquivo `teste_animacao_temperatura.py` instancia a rede e inicia o laboratório interativo. Os trechos apresentados abaixo são recortes essenciais do código, preservando sua lógica. Eles não substituem os arquivos-fonte completos. Quando houver

código de pós-processamento ou uma correção sugerida, isso será indicado no próprio texto.

B.1 Pacotes e organização

O módulo utiliza `matplotlib.pyplot` para a construção das figuras, `matplotlib` para mapas de cores, `FuncAnimation` e `matplotlib.widgets` para animação e controles interativos, além dos módulos `math`, `random`, `collections.deque` e `os`. A representação do grafo é implementada pela classe `Grafo`, baseada em dicionário de listas de adjacência.

O relatório principal já carrega `listings`, `xcolor` e `hyperref`, além dos pacotes matemáticos e gráficos. Portanto, os ambientes `lstlisting` utilizados neste apêndice são compatíveis com o preâmbulo apresentado no arquivo principal. Caso o relatório de destino ainda não contenha esses pacotes, adicionar ao preâmbulo:

```
\usepackage{xcolor}
\usepackage{listings}
```

A configuração de estilo Python definida no preâmbulo do relatório pode ser mantida.

B.2 Construção da rede com duas comunidades

A função `criar_grafo_comunidades(n,m,p,q,seed)` cria dois conjuntos disjuntos de vértices, percorre os pares possíveis dentro de cada conjunto e entre os conjuntos e decide a inclusão de cada aresta por meio de um gerador pseudoaleatório local. A função valida os tamanhos e as probabilidades, e armazena os grupos no atributo `grafo.grupos`.

Listing 1: Construção probabilística de uma rede com duas comunidades.

```
def criar_grafo_comunidades(n, m, p, q, seed=None):
    validar_n(n)
    validar_n(m)
    if not 0 <= p <= 1:
        raise ValueError("p deve estar entre 0 e 1.")
    if not 0 <= q <= 1:
        raise ValueError("q deve estar entre 0 e 1.")

    gerador = random.Random(seed)
    grafo = Grafo()
    grupo_a = list(range(n))
    grupo_b = list(range(n, n + m))

    for v in grupo_a + grupo_b:
        grafo.adicionar_vertice(v)

    for grupo in (grupo_a, grupo_b):
        for i in range(len(grupo)):
            for j in range(i + 1, len(grupo)):
                if gerador.random() < p:
                    grafo.adicionar_aresta(grupo[i], grupo[j])

```

```
    for u in grupo_a:
        for v in grupo_b:
            if gerador.random() < q:
                grafo.adicionar_aresta(u, v)

    grafo.grupos = {"A": grupo_a, "B": grupo_b}
    return grafo
```

A estrutura utiliza arestas não direcionadas e não permite laços. O método `adicionar_aresta` insere cada extremo na lista de adjacência do outro, evitando duplicações. A matriz de adjacência é simétrica e a matriz Laplaciana é obtida por $L = D - A$.

B.3 Parâmetros da execução de teste

O arquivo de teste fornecido contém a seguinte chamada:

Listing 2: Parâmetros definidos no script de teste.

```
grafo = criar_grafo_comunidades(
    n=10,
    m=10,
    p=0.80,
    q=0.05,
    seed=42
)

temperaturas = {v: 0.0 for v in grafo.adjacencia}

laboratorio_temperatura(
    grafo,
    temperaturas_iniciais=temperaturas,
    c=0.01,
    dt=1.0,
    passos=200,
    intervalo=100
)
```

A condição inicial passada pelo script é nula em todos os vértices. O laboratório também admite temperaturas iniciais omitidas, caso em que atribui zero a todos os vértices. A interface permite, posteriormente, selecionar um vértice e aplicar uma temperatura diferente por meio da caixa de texto. Essa alteração manual é necessária para produzir um gradiente térmico quando se parte da inicialização uniforme.

B.4 Inicialização e atualização da temperatura

O estado térmico é representado por um dicionário que associa cada vértice a um número real. A rotina `simular_troca_calor` valida se todos os vértices possuem temperatura e utiliza Euler explícito com atualização simultânea. Para cada vértice i, calcula-se

$$u_i^{k+1} = u_i^k + c\Delta t \sum_{j \sim i} (u_j^k - u_i^k).$$

A soma utiliza exclusivamente o estado u^k; o vetor completo u^{k+1} é construído antes de substituir o estado anterior.

Listing 3: Núcleo do método de Euler explícito com atualização simultânea.

```
for _ in range(passos):
    novas_temperaturas = {}

    for vertice in grafo.adjacencia:
        temperatura_atual = temperaturas[vertice]
        troca_de_calor = sum(
            temperaturas[vizinho] - temperatura_atual
            for vizinho in grafo.adjacencia[vertice]
        )
        novas_temperaturas[vertice] = (
            temperatura_atual + c * dt * troca_de_calor
        )

    temperaturas = novas_temperaturas
    historico.append(temperaturas.copy())
```

A função retorna uma lista de dicionários: o primeiro elemento é o estado inicial e o elemento de índice k corresponde ao estado após k passos. A implementação verifica $c \geq 0$, $\Delta t > 0$ e que o número de passos seja um inteiro não negativo.

O laboratório interativo contém uma segunda implementação do mesmo núcleo de atualização, que utiliza os valores atuais de c e Δt armazenados no estado da interface. A atualização é:

Listing 4: Atualização de um passo no laboratório interativo.

```
def passo():
    nonlocal temperaturas
    novas = {}
    for v in vertices:
        vizinhos = grafo.adjacencia[v]
        troca = sum(temperaturas[u] - temperaturas[v]
                    for u in vizinhos)
        novas[v] = temperaturas[v] + estado["c"] * estado["dt"
           ] * troca
    temperaturas = novas
    estado["passo"] += 1
    estado["historico_t"].append(estado["passo"] * estado["dt"
       ])
    registrar_media()
```

B.5 Método numérico e estabilidade

O método empregado é o método de Euler explícito aplicado ao sistema

$$\frac{du}{dt} = -cLu.$$

Em forma matricial, a atualização é

$$u^{k+1} = (I - c\Delta t L)u^k.$$

Como L é simétrica e semidefinida positiva para o grafo não direcionado, os autovalores da matriz de amplificação são $1 - c\Delta t\lambda_i$. Uma condição suficiente para estabilidade modal é

$$0 < c\Delta t\lambda_{\max}(L) < 2.$$

A implementação não calcula $\lambda_{\max}(L)$ e não verifica essa desigualdade antes da simulação. Assim, cabe ao usuário escolher parâmetros compatíveis com a rede. Para a realização produzida pelo script com semente 42, um pós-processamento independente da matriz Laplaciana fornece $\lambda_{\max} \approx 11{,}7200$. Esse valor pode ser utilizado para verificar parâmetros específicos, mas o cálculo espectral não faz parte do fluxo automatizado do programa.

B.6 Cálculo das médias comunitárias

A função auxiliar `temperatura_media_grupo` calcula a média aritmética das temperaturas de um conjunto de vértices:

Listing 5: Cálculo da média de temperatura de um grupo.

```
def temperatura_media_grupo(temperaturas, vertices):
    if not vertices:
        return 0.0
    return sum(temperaturas[v] for v in vertices) / len(
       vertices)
```

No laboratório interativo, as médias dos grupos são registradas em listas históricas separadas. O eixo temporal é atualizado pelo produto entre o número de passos e Δt. As curvas são desenhadas com rótulos "Grupo A" e "Grupo B".

Listing 6: Registro das médias das comunidades no laboratório.

```
def calc_media_grupo(grupo, dict_temps):
    if not grupo:
        return None
    return sum(dict_temps[v] for v in grupo) / len(grupo)

def registrar_media():
    a = calc_media_grupo(grupo_a, temperaturas)
    b = calc_media_grupo(grupo_b, temperaturas)
    if a is not None:
        estado["historico_a"].append(a)
    if b is not None:
        estado["historico_b"].append(b)
```

A interface não calcula diretamente a média global, a diferença absoluta entre as médias dos grupos ou a variância global. Essas grandezas podem ser calculadas a partir do dicionário de temperaturas, mas não devem ser descritas como métricas já produzidas pelo código original.

B.7 Critério de convergência implementado

O botão "Até Equilíbrio" interrompe a animação e avança a simulação enquanto a amplitude entre a maior e a menor temperatura for maior ou igual a 0,01, respeitando um limite de 5000 passos adicionais:

Listing 7: Laço de busca do estado aproximadamente homogêneo.

```
limite_max = 5000
candidato_passos = 0

while candidato_passos < limite_max:
    vals = list(temperaturas.values())
    if max(vals) - min(vals) < 0.01:
        break
    passo()
    candidato_passos += 1
```

A condição utilizada é, portanto,

$$\max_i u_i - \min_i u_i < 0{,}01.$$

O programa atualiza a informação visual com a mensagem "Equilíbrio atingido" após o laço, inclusive quando o limite de iterações é alcançado. Não há uma variável de retorno que informe se a condição foi satisfeita ou se o laço terminou apenas pelo limite. Para registrar um tempo de convergência confiável, deve-se acrescentar uma verificação explícita após o laço e armazenar o passo em que a condição foi satisfeita. O trecho abaixo ilustra essa correção. Trata-se de uma versão recomendada para distinguir a convergência efetiva do encerramento causado pelo limite de iterações; não corresponde ao comportamento do código original transcrito anteriormente.

Listing 8: Verificação explícita do resultado do critério de convergência.

```
limite_max = 5000
candidato_passos = 0
convergiu = False

while candidato_passos < limite_max:
    valores = list(temperaturas.values())

    if max(valores) - min(valores) < 0.01:
        convergiu = True
        break

    passo()
    candidato_passos += 1

# Verifica também o estado alcançado no último passo permitido.
valores = list(temperaturas.values())
convergiu = max(valores) - min(valores) < 0.01

if convergiu:
    info.set_text(
        f"Critério atingido em {candidato_passos} passos adicionais."
    )
else:
    info.set_text(
```

```
        f"Limite de {limite_max} passos atingido sem convergência."
    )
```

B.8 Métricas complementares para análise dos resultados

A implementação original registra as médias de temperatura dos grupos A e B, mas não calcula diretamente todas as métricas quantitativas que podem ser utilizadas na análise dos experimentos. O código a seguir apresenta funções simples de pós-processamento para obter a média global, a variância das temperaturas e a diferença absoluta entre as médias comunitárias. Essas funções são complementares e devem ser identificadas como acréscimos à análise, não como funcionalidades já presentes na interface original.

Listing 9: Cálculo complementar de métricas da distribuição térmica.

```
def temperatura_media(temperaturas):
    valores = list(temperaturas.values())
    if not valores:
        return 0.0
    return sum(valores) / len(valores)


def variancia_temperaturas(temperaturas):
    valores = list(temperaturas.values())
    if not valores:
        return 0.0

    media = sum(valores) / len(valores)
    return sum((valor - media) ** 2 for valor in valores) / len(valores)


def diferenca_medias_comunidades(temperaturas, grupo_a, grupo_b):
    media_a = temperatura_media_grupo(temperaturas, grupo_a)
    media_b = temperatura_media_grupo(temperaturas, grupo_b)
    return abs(media_a - media_b)
```

A variância mede a dispersão das temperaturas em torno da média global. À medida que a distribuição se torna homogênea, espera-se que essa grandeza diminua. Já a diferença entre as médias comunitárias permite acompanhar a aproximação térmica entre os dois grupos. Para uma rede conexa e uma dinâmica de difusão estável, ambas tendem a zero no equilíbrio.

O tempo de convergência pode ser extraído do histórico da simulação procurando o primeiro estado que satisfaça o critério de amplitude. O exemplo seguinte pressupõe que `historico` seja a lista de estados produzida pela rotina de simulação, na qual cada estado é um dicionário de temperaturas:

Listing 10: Determinação do primeiro instante que satisfaz o critério de amplitude.

```
def primeiro_passo_convergente(historico, tolerancia=0.01):
```

```
    for indice, estado in enumerate(historico):
        valores = list(estado.values())

        if valores and max(valores) - min(valores) <
           tolerancia:
            return indice

    return None
```

O retorno `None` indica que nenhum estado armazenado satisfez a tolerância. Se o passo temporal for constante e igual a Δt, o instante aproximado correspondente ao índice retornado é $t = k\Delta t$. Essas métricas permitem comparar experimentos, desde que sejam calculadas com o mesmo critério de convergência e os mesmos parâmetros de tolerância.

B.9 Propriedades espectrais

A classe `Grafo` implementa a matriz de adjacência, a matriz diagonal de graus e a matriz Laplaciana combinatória. A definição computacional é $L = D - A$. Entretanto, o módulo não contém chamadas a rotinas de autovalores, como `numpy.linalg.eigvalsh`, nem funções para calcular λ_2, $\lambda_{\max}$ ou autovetores. Os valores espectrais informados no corpo do relatório foram obtidos por pós-processamento numérico da realização gerada pelo próprio algoritmo e devem ser identificados como tal.

Para reproduzir esse pós-processamento fora do módulo principal, pode-se utilizar o seguinte procedimento:

Listing 11: Pós-processamento espectral da matriz Laplaciana.

```
import numpy as np

vertices, matriz_a = grafo.matriz_adjacencia()
A = np.asarray(matriz_a, dtype=float)
graus = A.sum(axis=1)
L = np.diag(graus) - A

autovalores = np.linalg.eigvalsh(L)
lambda_2 = autovalores[1]
lambda_max = autovalores[-1]
```

Esse trecho é um procedimento de análise complementar, não uma função existente no código entregue. Como a matriz é simétrica, `eigvalsh` é apropriada para obter seus autovalores reais.

B.10 Implementação dos controles e da animação

A interface interativa foi implementada com os componentes do módulo `matplotlib.widgets`. A caixa de texto permite informar a temperatura a ser atribuída ao vértice selecionado, enquanto os botões controlam a execução, a pausa, a reinicialização e a execução até o critério de equilíbrio. Os controles deslizantes permitem modificar o número máximo de passos, a constante de difusão c, o passo temporal Δt e o intervalo de atualização da animação.

O trecho de código a seguir apresenta a criação dos controles, os principais procedimentos de interação e a chamada de `FuncAnimation`. Ele deve ser inserido em conjunto com as definições de estado, variáveis, eixos e funções auxiliares da rotina `laboratorio_temperatura`.

Listing 12: Implementação dos controles e da animação do laboratório de difusão

```
# Criacao dos controles
ax_temp = fig.add_axes([0.18, 0.22, 0.08, 0.038])
caixa_temp = widgets.TextBox(ax_temp, "Temperatura: ", initial
    ="100")

ax_aplicar = fig.add_axes([0.27, 0.22, 0.09, 0.038])
botao_aplicar = widgets.Button(ax_aplicar, "Aplicar")

ax_play = fig.add_axes([0.38, 0.22, 0.09, 0.038])
botao_play = widgets.Button(ax_play, "Play")

ax_reset = fig.add_axes([0.49, 0.22, 0.09, 0.038])
botao_reset = widgets.Button(ax_reset, "Reiniciar")

ax_equil = fig.add_axes([0.60, 0.22, 0.13, 0.038])
botao_equil = widgets.Button(ax_equil, "Ate Equilibrio")

# Controles deslizantes
ax_passos = fig.add_axes([0.18, 0.14, 0.26, 0.025])
slider_passos = widgets.Slider(
    ax_passos, "Tempo (Passos)",
    10, 2000, valinit=passos, valstep=10
)

ax_c = fig.add_axes([0.54, 0.14, 0.26, 0.025])
slider_c = widgets.Slider(
    ax_c, "c", 0.0, 0.10,
    valinit=c, valstep=0.001
)

ax_dt = fig.add_axes([0.18, 0.085, 0.26, 0.025])
slider_dt = widgets.Slider(
    ax_dt, "dt", 0.01, 10.0,
    valinit=dt, valstep=0.01
)

ax_vel = fig.add_axes([0.54, 0.085, 0.26, 0.025])
slider_vel = widgets.Slider(
    ax_vel, "Velocidade", 20, 500,
    valinit=intervalo, valstep=10
)

# Atualizacao de um passo da simulacao
def passo():
    nonlocal temperaturas
```

```
45
46      novas = {}
47
48      for v in vertices:
49          vizinhos = grafo.adjacencia[v]
50          troca = sum(
51              temperaturas[u] - temperaturas[v]
52              for u in vizinhos
53          )
54
55          novas[v] = (
56              temperaturas[v]
57              + estado["c"] * estado["dt"] * troca
58          )
59
60      temperaturas = novas
61      estado["passo"] += 1
62      estado["historico_t"].append(
63          estado["passo"] * estado["dt"]
64      )
65
66      registrar_media()
67
68
69  # Selecao de vertice pelo clique
70  def clicar(event):
71      if event.inaxes != ax or event.xdata is None or event.
           ydata is None:
72          return
73
74      menor = None
75      menor_dist = float("inf")
76
77      for v in vertices:
78          x, y = posicoes[v]
79          d = (
80              (event.xdata - x) ** 2
81              + (event.ydata - y) ** 2
82          )
83
84          if d < menor_dist:
85              menor_dist = d
86              menor = v
87
88      if menor is not None and menor_dist < 0.10:
89          estado["selecionado"] = menor
90          caixa_temp.set_val(f"{temperaturas[menor]:.1f}")
91          info.set_text(f"Vertice {menor} selecionado.")
92          atualizar_visual()
93
94
```

```
95  # Aplicacao de temperatura ao vertice selecionado
96  def aplicar(event):
97      nonlocal t_min_fixo, t_max_fixo, norm_fixa
98
99      v = estado["selecionado"]
100
101     if v is None:
102         info.set_text("Selecione primeiro um vertice no grafo.
                ")
103         return
104
105     try:
106         valor = float(caixa_temp.text)
107     except ValueError:
108         info.set_text("Digite um numero valido.")
109         return
110
111     temperaturas[v] = valor
112     temperaturas_reset[v] = valor
113
114     m_a = calc_media_grupo(grupo_a, temperaturas)
115     m_b = calc_media_grupo(grupo_b, temperaturas)
116     m_validas = [m for m in [m_a, m_b] if m is not None]
117
118     if m_validas:
119         t_max_fixo = max(m_validas)
120         t_min_fixo = min(m_validas)
121
122         if abs(t_max_fixo - t_min_fixo) < 1e-4:
123             t_min_fixo = max(0.0, t_min_fixo - 0.5)
124             t_max_fixo += 0.5
125
126         norm_fixa = mpl.colors.Normalize(
127             vmin=t_min_fixo, vmax=t_max_fixo
128         )
129         scatter.set_norm(norm_fixa)
130         sm.set_norm(norm_fixa)
131         colorbar.update_normal(sm)
132
133     info.set_text(
134         f"Vertice {v} alterado para {valor:.2f} graus."
135     )
136     atualizar_visual()
137
138
139 # Controle Play/Pause
140 def play_pause(event):
141     estado["rodando"] = not estado["rodando"]
142     botao_play.label.set_text(
143         "Pause" if estado["rodando"] else "Play"
144     )
```

```
145
146
147 # Restauracao da distribuicao salva
148 def reiniciar(event):
149     nonlocal temperaturas
150
151     temperaturas = temperaturas_reset.copy()
152     estado["rodando"] = False
153     estado["passo"] = 0
154     estado["historico_t"] = [0.0]
155     estado["historico_a"] = []
156     estado["historico_b"] = []
157
158     botao_play.label.set_text("Play")
159     registrar_media()
160     atualizar_visual()
161
162
163 # Atualizacao dos parametros dos controles deslizantes
164 def mudar_passos(val):
165     estado["max_passos"] = int(val)
166     atualizar_visual()
167
168
169 def mudar_dt(val):
170     estado["dt"] = float(val)
171     atualizar_visual()
172
173
174 def mudar_velocidade(val):
175     anim.event_source.interval = int(val)
176
177
178 # Execucao ate o criterio de amplitude
179 def ir_ate_equilibrio(event):
180     estado["rodando"] = False
181     botao_play.label.set_text("Play")
182
183     limite_max = 5000
184     candidato_passos = 0
185
186     while candidato_passos < limite_max:
187         valores = list(temperaturas.values())
188
189         if max(valores) - min(valores) < 0.01:
190             break
191
192         passo()
193         candidato_passos += 1
194
195     estado["max_passos"] = max(
```

```
196             estado["passo"], estado["max_passos"]
197         )
198         slider_passos.set_val(estado["max_passos"])
199 
200         info.set_text(
201             f"Equilibrio atingido em {estado['passo']} passos!"
202         )
203         atualizar_visual()
204 
205 
206 # Atualizacao periodica da animacao
207 def animar(frame):
208         if not estado["rodando"]:
209             return
210 
211         if estado["passo"] >= estado["max_passos"]:
212             estado["rodando"] = False
213             botao_play.label.set_text("Play")
214             return
215 
216         passo()
217         atualizar_visual()
218 
219 
220 # Associacao dos eventos aos controles
221 fig.canvas.mpl_connect("button_press_event", clicar)
222 
223 botao_aplicar.on_clicked(aplicar)
224 botao_play.on_clicked(play_pause)
225 botao_reset.on_clicked(reiniciar)
226 botao_equil.on_clicked(ir_ate_equilibrio)
227 
228 slider_passos.on_changed(mudar_passos)
229 slider_c.on_changed(
230         lambda val: estado.update({"c": float(val)})
231 )
232 slider_dt.on_changed(mudar_dt)
233 slider_vel.on_changed(mudar_velocidade)
234 
235 # Criacao e manutencao da animacao
236 anim = FuncAnimation(
237         fig,
238         animar,
239         interval=intervalo,
240         cache_frame_data=False
241 )
242 
243 fig._laboratorio_temperatura = anim
244 plt.show()
```

A rotina `passo()` implementa a atualização explícita da temperatura em cada vértice, utilizando as temperaturas dos seus vizinhos. A seleção de um vértice é

realizada pela posição do clique no sistema de coordenadas do desenho. Após a seleção, a caixa de texto pode alterar sua temperatura, e o valor aplicado também é armazenado para ser recuperado pelo botão *Reiniciar*.

O botão *Até Equilíbrio* interrompe a animação e executa sucessivos passos da simulação até que a diferença entre a maior e a menor temperatura da rede seja inferior a 0,01, ou até que sejam realizados 5000 passos adicionais nessa execução. Portanto, o limite de iterações também funciona como uma proteção contra execuções excessivamente longas; atingir esse limite não garante, por si só, que o equilíbrio tenha sido alcançado.

Por fim, o parâmetro `intervalo` de `FuncAnimation` é expresso em milissegundos e regula a frequência de atualização visual. Ele não corresponde ao passo temporal físico Δt, que aparece na equação numérica de atualização das temperaturas.

C Exemplos passo a passo dos algoritmos e da dinâmica

Esta seção acrescenta exemplos pequenos e verificáveis para tornar mais explícita a execução dos algoritmos e a interpretação dos objetos matemáticos utilizados no trabalho. Os exemplos de busca e de Kruskal são ilustrativos: servem para explicar os procedimentos e não representam, necessariamente, a rede aleatória utilizada nos experimentos de difusão.

C.1 Busca em largura (BFS): execução passo a passo

Considere o grafo não direcionado com vértices $V = \{A, B, C, D, E, F\}$ e arestas

$$E = \{\{A, B\}, \{A, C\}, \{B, D\}, \{B, E\}, \{C, F\}\}.$$

A disposição esquemática abaixo representa as conexões:

```
  A
 / \
 B C
/ \\
D E F
```

A busca começa em A. Supondo que os vizinhos sejam examinados na ordem alfabética, a fila (FIFO) evolui conforme a Tabela 2. Um vértice é marcado como visitado no momento em que é inserido na fila, evitando inserções repetidas.

A ordem de visita é

$$A,\ B,\ C,\ D,\ E,\ F.$$

As distâncias a partir de A, medidas em número de arestas, são

$$d(A) = 0, \quad d(B) = d(C) = 1, \quad d(D) = d(E) = d(F) = 2.$$

A BFS visita primeiro todos os vértices a uma distância 1, depois os vértices a distância 2, e assim sucessivamente. Por isso, em grafos não ponderados, ela encontra caminhos com o menor número de arestas.

Tabela 2: Execução da BFS a partir do vértice A.

Etapa	Vértice removido	Novos vértices inseridos	Fila após a etapa
Inicial	–	A	$[A]$
1	A	B, C	$[B, C]$
2	B	D, E	$[C, D, E]$
3	C	F	$[D, E, F]$
4	D	–	$[E, F]$
5	E	–	$[F]$
6	F	–	[]

C.2 Busca em profundidade (DFS): execução passo a passo

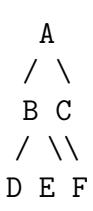
No mesmo grafo, a DFS explora um ramo até não encontrar novos vértices e, então, retorna ao vértice anterior para continuar a busca. Considerando novamente a ordem alfabética dos vizinhos, a execução recursiva é:

1. Visita A e segue para B.
2. A partir de B, segue para D. Como D não possui vizinho ainda não visitado, retorna a B.
3. De B, segue para E. Como não há novo vizinho, retorna a B e depois a A.
4. De A, segue para C e, em seguida, para F.
5. Como F não possui vizinho novo, a busca retorna e termina.

A ordem de descoberta é

$$A,\ B,\ D,\ E,\ C,\ F.$$

A árvore de busca resultante pode ser representada por:



Neste exemplo, a árvore de busca tem a mesma forma das conexões do grafo. Em grafos com arestas adicionais, a árvore registra apenas as arestas pelas quais os vértices foram descobertos. A ordem da DFS depende da ordem em que os vizinhos são percorridos; portanto, outra ordenação das listas de adjacência pode produzir uma ordem diferente, sem alterar a validade da busca.

C.3 Algoritmo de Kruskal: exemplo completo com pesos

O algoritmo de Kruskal constrói uma árvore geradora mínima de um grafo conexo e ponderado. Para isso, ordena as arestas em ordem crescente de peso e adiciona uma aresta somente quando ela conecta componentes distintas. A condição evita a formação de ciclos.

Considere o grafo com vértices $V = \{A, B, C, D, E\}$ e as seguintes arestas ponderadas:

$$w(A,B) = 1, \quad w(B,C) = 2, \quad w(A,C) = 3,$$
$$w(B,D) = 4, \quad w(C,D) = 5, \quad w(C,E) = 6,$$
$$w(D,E) = 7.$$

A ordem crescente das arestas é

$$(A,B),\ (B,C),\ (A,C),\ (B,D),\ (C,D),\ (C,E),\ (D,E).$$

A execução completa está na Tabela 3.

Tabela 3: Execução do algoritmo de Kruskal no exemplo ponderado.

Aresta	Peso	Decisão	Justificativa
(A,B)	1	Aceita	Une A e B
(B,C)	2	Aceita	Une C à componente AB
(A,C)	3	Rejeita	Formaria o ciclo $A-B-C-A$
(B,D)	4	Aceita	Une D à componente ABC
(C,D)	5	Rejeita	C e D já estão conectados
(C,E)	6	Aceita	Une E à componente $ABCD$
(D,E)	7	Rejeita	D e E já estão conectados

Ao final, a árvore geradora mínima contém as arestas

$$T = \{(A,B), (B,C), (B,D), (C,E)\}$$

e seu peso total é

$$w(T) = 1 + 2 + 4 + 6 = 13.$$

Como há cinco vértices, uma árvore geradora possui exatamente quatro arestas. O procedimento termina quando todos os vértices pertencem à mesma componente. A implementação eficiente utiliza a estrutura *Union-Find* (ou conjuntos disjuntos) para verificar se os extremos de uma aresta já pertencem à mesma componente.

Este exemplo documenta o funcionamento matemático de Kruskal. Como delimitado anteriormente, o código-fonte disponibilizado para o experimento de difusão não contém uma implementação de Kruskal; portanto, o exemplo não deve ser apresentado como resultado de uma execução desse programa.

C.4 Matriz Laplaciana e interpretação dos autovetores

Para um grafo não direcionado, a matriz Laplaciana combinatória é definida por

$$L = D - A,$$

em que A é a matriz de adjacência e D é a matriz diagonal cujos elementos são os graus dos vértices. A Laplaciana é simétrica e semidefinida positiva. Assim, seus autovalores são reais e não negativos.

Se $Lv_i = \lambda_i v_i$, o autovetor v_i descreve um padrão de variação associado ao autovalor λ_i. Na dinâmica de difusão

$$\frac{du}{dt} = -cLu,$$

cada componente do estado na direção de um autovetor evolui de forma independente, sendo multiplicada continuamente por

$$e^{-c\lambda_i t}.$$

Consequentemente:

- O autovetor associado a $\lambda_1 = 0$, em um grafo conexo, é proporcional ao vetor constante $\mathbf{1}$. Essa componente representa a média global, que é preservada pela difusão.
- Autovetores associados a autovalores pequenos representam padrões que decaem lentamente. Eles descrevem variações que a rede tem maior dificuldade de eliminar.
- Autovetores associados a autovalores maiores representam padrões mais oscilatórios entre vértices conectados e decaem mais rapidamente no modelo contínuo.

Os autovetores não devem ser interpretados isoladamente como rótulos automáticos de comunidades. Sua interpretação depende dos valores assumidos em cada vértice e da estrutura do grafo. Em particular, o autovetor associado ao segundo menor autovalor, chamado vetor de Fiedler, é útil para investigar separações estruturais: valores com sinais opostos ou magnitudes distintas podem indicar grupos de vértices fracamente conectados entre si.

C.5 Conectividade algébrica e separação entre comunidades

Ordenando os autovalores da Laplaciana,

$$0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n,$$

o valor λ_2 é denominado conectividade algébrica. Para um grafo não direcionado, vale:

$$\lambda_2 > 0 \quad \Longleftrightarrow \quad \text{o grafo é conexo.}$$

Se o grafo é desconexo, existem pelo menos dois autovalores nulos e, portanto, $\lambda_2 = 0$.

A conectividade algébrica também está relacionada à presença de gargalos na rede. Quando dois grupos possuem muitas arestas internas e poucas arestas entre si, a ligação entre as comunidades é fraca. Nesse cenário, é comum obter um λ_2 pequeno. O vetor de Fiedler tende a apresentar valores relativamente semelhantes dentro de cada grupo e uma mudança mais acentuada entre os grupos, oferecendo uma indicação espectral de possível separação comunitária.

Para a dinâmica de difusão, essa interpretação tem consequência direta. A solução pode ser decomposta nos modos próprios da Laplaciana:

$$u(t) = \sum_{i=1}^{n} a_i e^{-c\lambda_i t} v_i.$$

Após a componente constante, o modo associado a λ_2 é o que decai mais lentamente, desde que sua amplitude inicial não seja nula. Assim, uma estimativa da escala temporal dominante de convergência é

$$\tau_2 \approx \frac{1}{c\lambda_2}.$$

Um valor menor de λ_2 indica, em geral, uma aproximação mais lenta ao equilíbrio, pois o modo mais lento permanece significativo durante mais tempo. Um valor maior indica maior conectividade global e uma dissipação mais rápida desse modo. Essa relação deve ser analisada mantendo-se comparáveis o tamanho da rede, a distribuição dos graus e os demais parâmetros.

No experimento com duas comunidades, os parâmetros p e q controlam, respectivamente, as probabilidades de conexão dentro e entre os grupos. Com p elevado e q reduzido, espera-se uma estrutura com maior densidade interna e menor densidade entre comunidades. A interpretação de λ_2 permite relacionar essa estrutura à velocidade da homogeneização térmica. A quantidade de arestas entre os grupos não determina sozinha o valor de λ_2: a posição dessas arestas e a distribuição dos graus também influenciam a conectividade algébrica.

C.6 Euler explícito: exemplo numérico de uma etapa

Considere o caminho com três vértices $1 - 2 - 3$. Sua matriz Laplaciana é

$$L = \begin{bmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{bmatrix}.$$

A dinâmica de difusão é

$$\frac{du}{dt} = -cLu.$$

Pelo método de Euler explícito, a aproximação em um passo é

$$u^{k+1} = (I - c\Delta t L)u^k.$$

Escolhendo $c = 0{,}1$, $\Delta t = 1$ e condição inicial

$$u^0 = \begin{bmatrix} 100 \\ 0 \\ 0 \end{bmatrix},$$

temos

$$Lu^0 = \begin{bmatrix} 100 \\ -100 \\ 0 \end{bmatrix}.$$

Logo,

$$\begin{aligned} u^1 &= u^0 - 0{,}1 \begin{bmatrix} 100 \\ -100 \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} 90 \\ 10 \\ 0 \end{bmatrix}. \end{aligned}$$

Após uma etapa, o vértice 1 perde 10 unidades de temperatura, o vértice 2 recebe 10 unidades e o vértice 3 permanece inicialmente em zero. A soma das temperaturas é preservada:

$$100 + 0 + 0 = 90 + 10 + 0 = 100.$$

Esse resultado ilustra a conservação da média global em uma rede não direcionada, desde que a atualização seja realizada simultaneamente a partir do estado anterior.

C.7 Estabilidade e interpretação dos fatores de amplificação

Seja λ_i um autovalor da Laplaciana. Ao aplicar Euler explícito ao modo próprio correspondente, sua amplitude é multiplicada a cada passo pelo fator

$$g_i = 1 - c\Delta t \lambda_i.$$

Esse valor é denominado fator de amplificação do modo. Para que o modo não cresça numericamente, é necessário que

$$|g_i| \leq 1.$$

Para os modos não constantes, uma condição suficiente usual de estabilidade é

$$0 < c\Delta t \lambda_{\max}(L) < 2.$$

A condição deve ser verificada para o maior autovalor, pois ele produz o fator mais restritivo.

A interpretação dos fatores é:

- $0 < g_i < 1$: a amplitude mantém o sinal e diminui a cada passo;
- $-1 < g_i < 0$: a amplitude diminui, mas alterna de sinal entre passos, produzindo oscilações numéricas amortecidas;
- $g_i = 0$: o modo é eliminado em uma única atualização;
- $|g_i| > 1$: a amplitude cresce em módulo, caracterizando instabilidade para esse modo.

O modo constante possui $\lambda_1 = 0$ e, portanto, $g_1 = 1$. Isso não representa instabilidade: corresponde à preservação da média global da temperatura.

A Tabela 4 apresenta uma leitura numérica ilustrativa para $c\Delta t\lambda_i = 0{,}2$, 1, 1,5 e 2,2.

Tabela 4: Interpretação dos fatores de amplificação.

$c\Delta t\lambda_i$	$g_i = 1 - c\Delta t\lambda_i$	Comportamento
0,2	0,8	Decaimento sem troca de sinal
1,0	0	Modo eliminado em um passo
1,5	$-0{,}5$	Decaimento com alternância de sinal
2,2	$-1{,}2$	Crescimento em módulo; instável

A evolução do módulo do fator pode ser visualizada esquematicamente pela sequência de amplitudes $1, g_i, g_i^2, g_i^3, \ldots$. Para $g_i = 0{,}8$, por exemplo, obtém-se

$$1, \quad 0{,}8, \quad 0{,}64, \quad 0{,}512, \quad 0{,}4096, \ldots$$

e o modo decai progressivamente. Para $g_i = -0{,}5$, a sequência é

$$1, \quad -0{,}5, \quad 0{,}25, \quad -0{,}125, \quad 0{,}0625, \ldots,$$

mostrando alternância de sinal com redução da amplitude. Já para $g_i = -1{,}2$, a sequência

$$1, \quad -1{,}2, \quad 1{,}44, \quad -1{,}728, \ldots$$

cresce em módulo, evidenciando a instabilidade.

A representação a seguir resume graficamente os três comportamentos para a amplitude em função do número de passos:

Passo k	0	1	2	3	4
$g = 0{,}8$	1	0,8	0,64	0,512	0,4096
$g = -0{,}5$	1	-0,5	0,25	-0,125	0,0625
$g = -1{,}2$	1	-1,2	1,44	-1,728	2,0736

Portanto, reduzir o passo temporal Δt ou a constante c reduz o produto $c\Delta t$ e pode tornar a atualização estável. A escolha dos parâmetros deve considerar $\lambda_{\max}(L)$, que depende da estrutura do grafo. No código original descrito neste apêndice, essa verificação não é realizada automaticamente; cabe ao usuário selecionar parâmetros compatíveis com a rede ou acrescentar uma rotina de validação antes de iniciar a simulação.